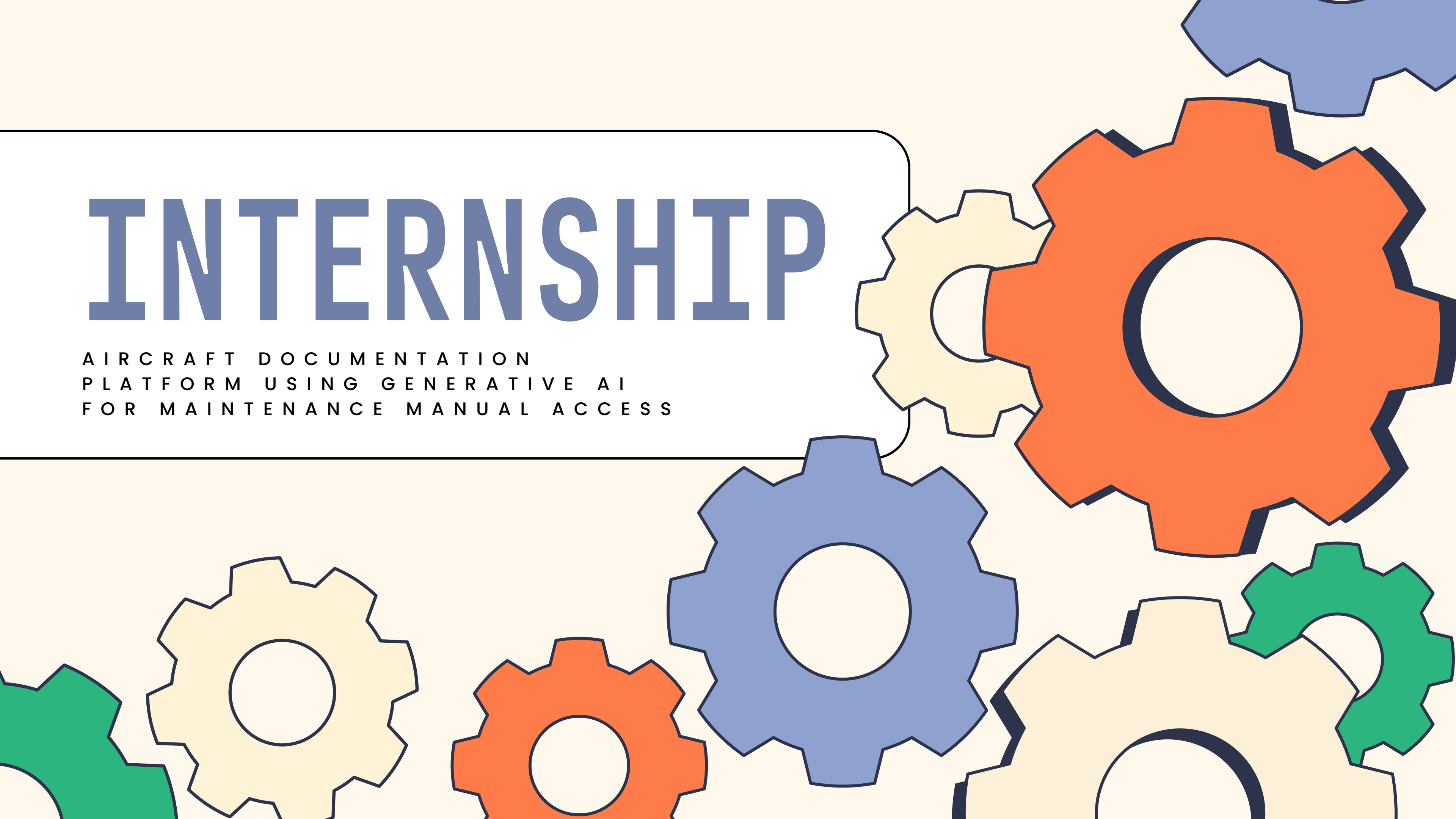


INTERNSHIP

AIRCRAFT DOCUMENTATION
PLATFORM USING GENERATIVE AI
FOR MAINTENANCE MANUAL ACCESS



COMPANY BACKGROUND

AVOTRON PTE. LTD.



- Incorporated on 14 August 2008 (Thursday) in Singapore.
- Main activity is to manufacture and repair of engineering and scientific instruments.
- Secondary activity is to perform wholesale trade of a variety of goods without a dominant product.

GOVERNMENT PROCUREMENT

- Installation, Testing and Commissioning for Asset Management System for Singapore Polytechnic (Awarded in 15 Oct 2024)
- Supply, Delivery, Installation and Commissioning of the Aircraft Cockpit Trainer Instrument System Module for Singapore Polytechnic (Awarded in 6 May 2025)

● AWARDED

PUBLISHED
09 Sep 2024

CLOSING
20 Sep 2024

SPO-REQ-2024-006803-EEE-Supply, Installation, Testing and Commissioning for Asset Management System

Ref: SPO000ETQ24000156

AGENCY	AWARDED	AWARD VALUE
Singapore Polytechnic	15 Oct 2024	\$89,500.00

● AWARDED

PUBLISHED
23 Jan 2025

CLOSING
05 Feb 2025

SPO-REQ-2025-000449-EEE-Supply, Delivery, Installation, and Commissioning of the Aircraft Cockpit Trainer for Aircraft Instrument System Module

Ref: SPO000ETQ25000021

AGENCY	AWARDED	AWARD VALUE
Singapore Polytechnic	06 May 2025	\$63,800.00

**AIRCRAFT DOCUMENTATION PLATFORM
USING GENERATIVE AI FOR MAINTENANCE
MANUAL ACCESS**

• S Y P N O S I S

A web-based dashboard for aircraft maintenance specialists to efficiently locate relevant sections of technical manuals based on system or component queries. Thus, locating the exact manual and relevant sections corresponding to a specific system or component can be less time consuming and be more efficient allowing faster maintenance process.

• C O N C E I V E

To provide users a way to quickly find, understand, and act on the right information from dense aviation manuals and also be locally hosted for security purposes as manuals are classified. A scalable RAG pipeline to allow different types of manuals to be integrated.

• D E S I G N

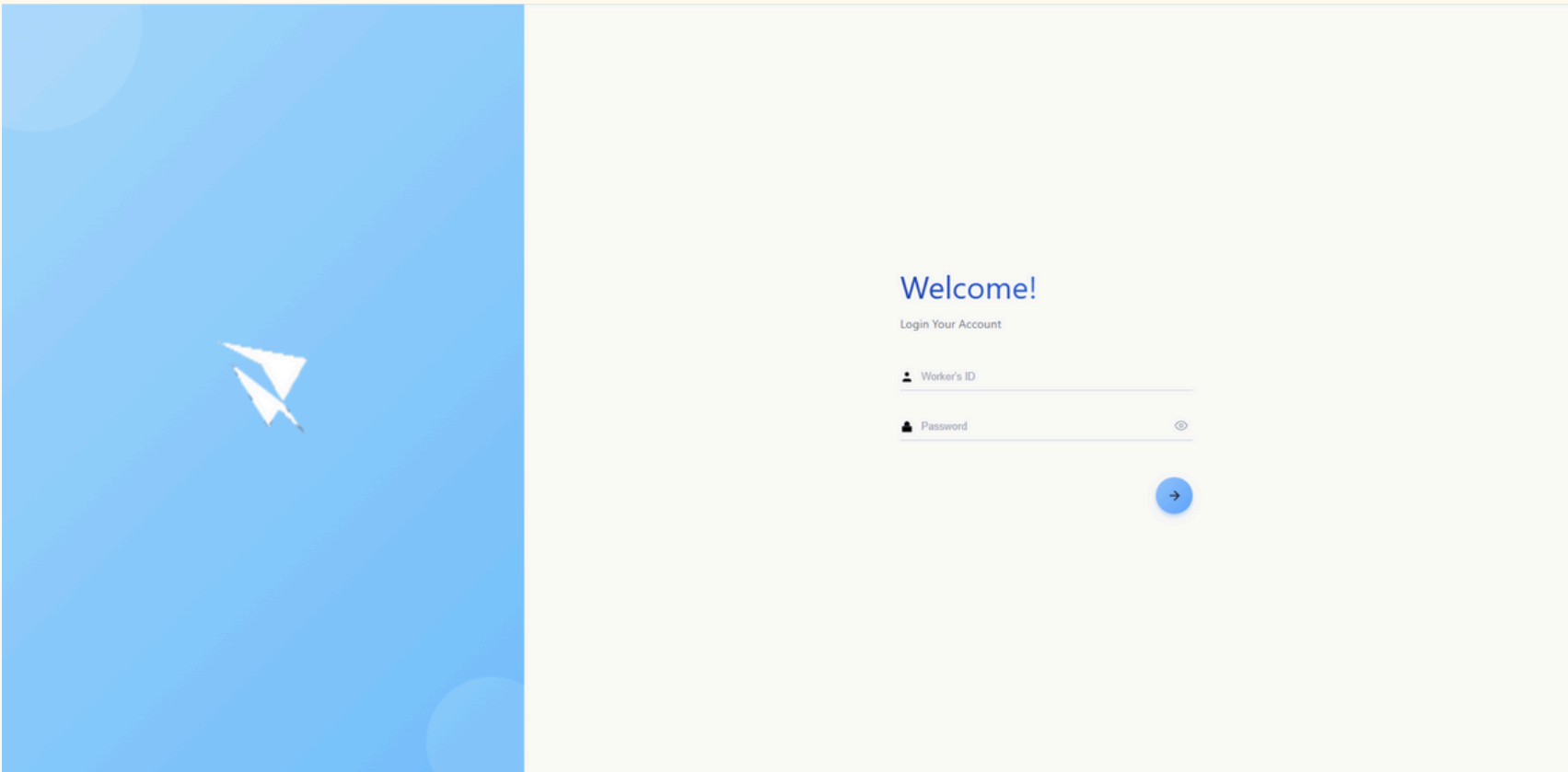
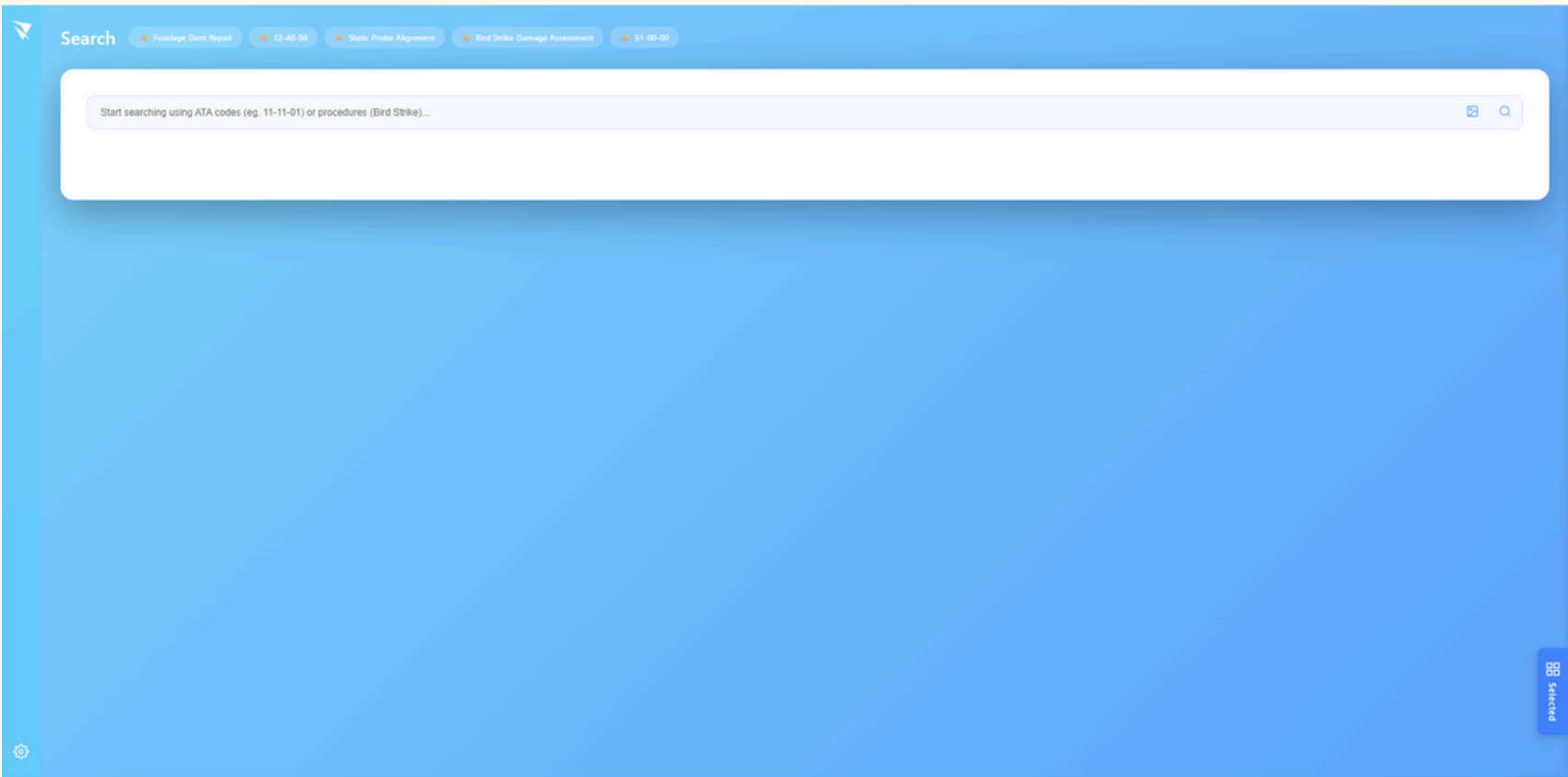
- Python - Backend
- HTML/CSS/JS - Frontend
- HuggingFace(all-MiniLM-L6-v2) - Embedding Model
- Ollama(Phi:14b/qwen2.5-vl-7b) - Large/Vision Language Model (LLM/VLM)
- MinIO - Blob Storage for Files
- Qdrant - Vector Database
- DuckDB - Database for website to be persistent

• I M P L E M E N T

Stages - Research, Design, Experiment, Prototype, Adjust/Improve

• O P E R A T E

Test prototype’s accuracy before deploying



P R O B L E M

- **BIGGEST PROBLEMS FACED BY MAINTENANCE ENGINEERS IN 2025 - QOCO.AERO, 2LYONS**

- **Shortage of Skilled Maintenance Technicians** - Leading to, increased reliance on overtime and potential errors.
- **Aging Fleet and Maintenance Complexity** - Require more frequent inspections, repairs, and part replacements.
- **Unplanned Downtime** - Unexpected failures or maintenance needs often lead to unplanned downtime, which is a significant challenge in aviation maintenance.

- **BEST SOLUTIONS?**

- **Predictive Maintenance** - Airlines and maintenance organizations are investing in predictive maintenance systems. These systems use data analytics and sensors to predict potential failures before they occur, enabling teams to act proactively.
- **Consolidated Knowledge** - Alongside predictive maintenance, there is a growing emphasis on consolidating knowledge across manuals, systems, and operational data. Instead of relying on fragmented sources, organizations are building integrated platforms that unify technical documentation, maintenance records, and AI-powered search tools.

CONCEIVE

- **WHAT ARE THE USER’S NEEDS?**

A web application to quickly find, understand, and act on the right information from dense aviation manuals and also be locally hosted for security purposes as manuals are classified.

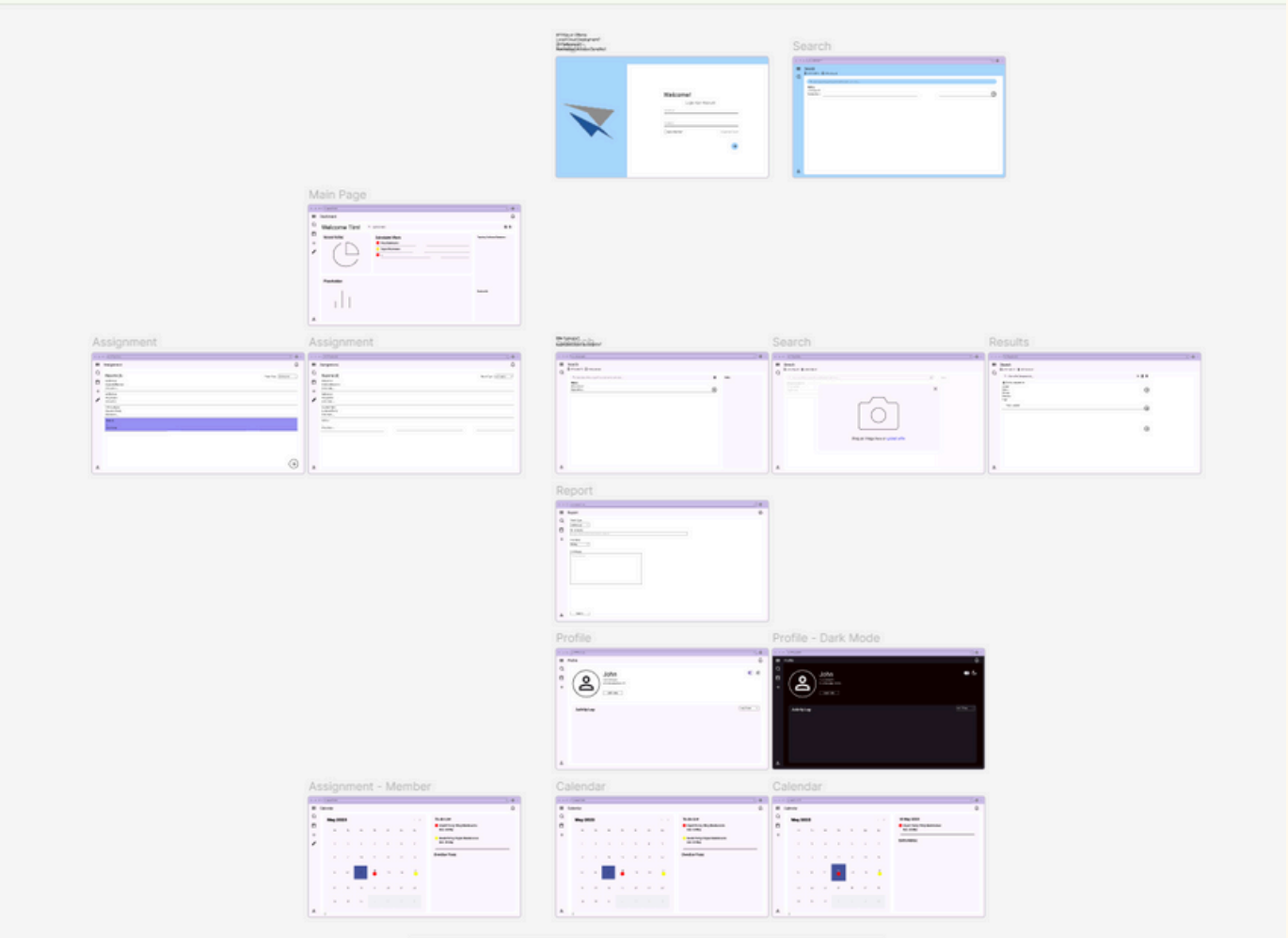
PROBLEM STATEMENT:

Aircraft maintenance specialists face inefficiencies in locating relevant information within vast technical manuals, leading to delays, increased workload, and potential risks to operational safety.

- **WHAT FEATURES WILL HELP MEET THESE NEEDS TO ACHIEVE THE FUNCTIONAL GOALS OF THE PROJECT?**

- Natural Language Search for RAG
- Quick Retrieval of Manuals
- Web Application
- Chat History
- Temporary Saving of ATA Codes List
- Locally Hosted (Offline)
- Extraction of ATA Codes
- AI Suggestions
- Audit Log (Security & Data Purposes)
- Damage Assessment through Image Search

- AI Summarisation
- Multiple Machine Accessibility
- File Uploading for scalability
- Login/Logout



SOMA SOFTWARE



Centralized dashboard for all aircraft documents (manuals, maintenance records, certifications).

PROS:

- Automated expiration tracking for compliance.
- Real-time visibility into document status.
- Aircraft alerts & notifications.

CONS:

It's primarily a compliance and tracking tool and storage, not designed for deep parsing. No usage of Gen AI to help with user experience.

MICROSOFT 365



Microsoft 365

AI powered search/finding of documents called Copilot Search (Microsoft's Gen AI).

PROS:

- Unified Search Across Apps (Word, OneDrive, Teams)
- Natural Language Search
- Semantic Understanding
- Anything a Gen AI can do

CONS:

It's optimized for finding documents, but not extracting specific procedures often requiring multiple prompts and lacking domain-specific accuracy.

DESIGN

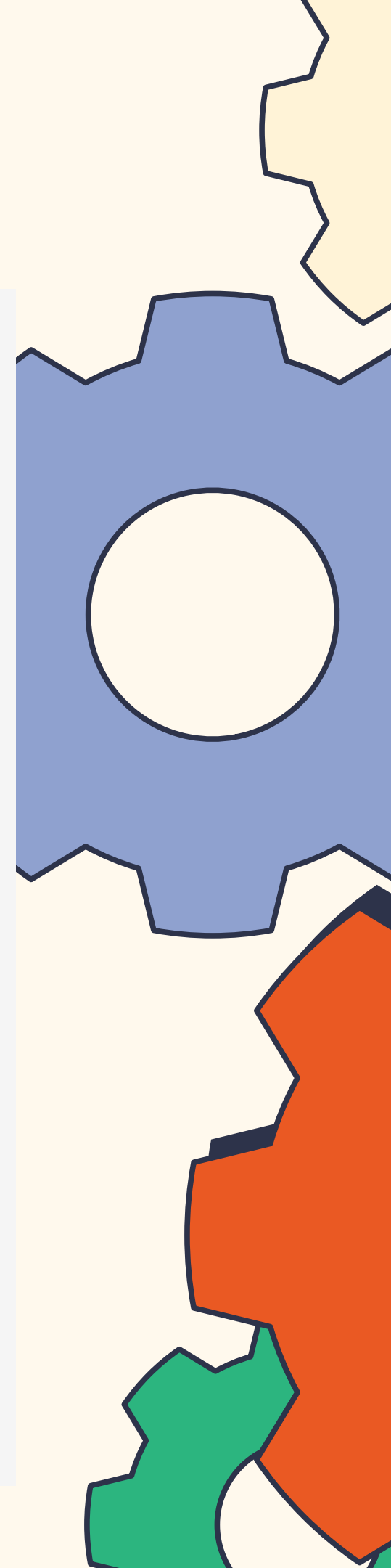
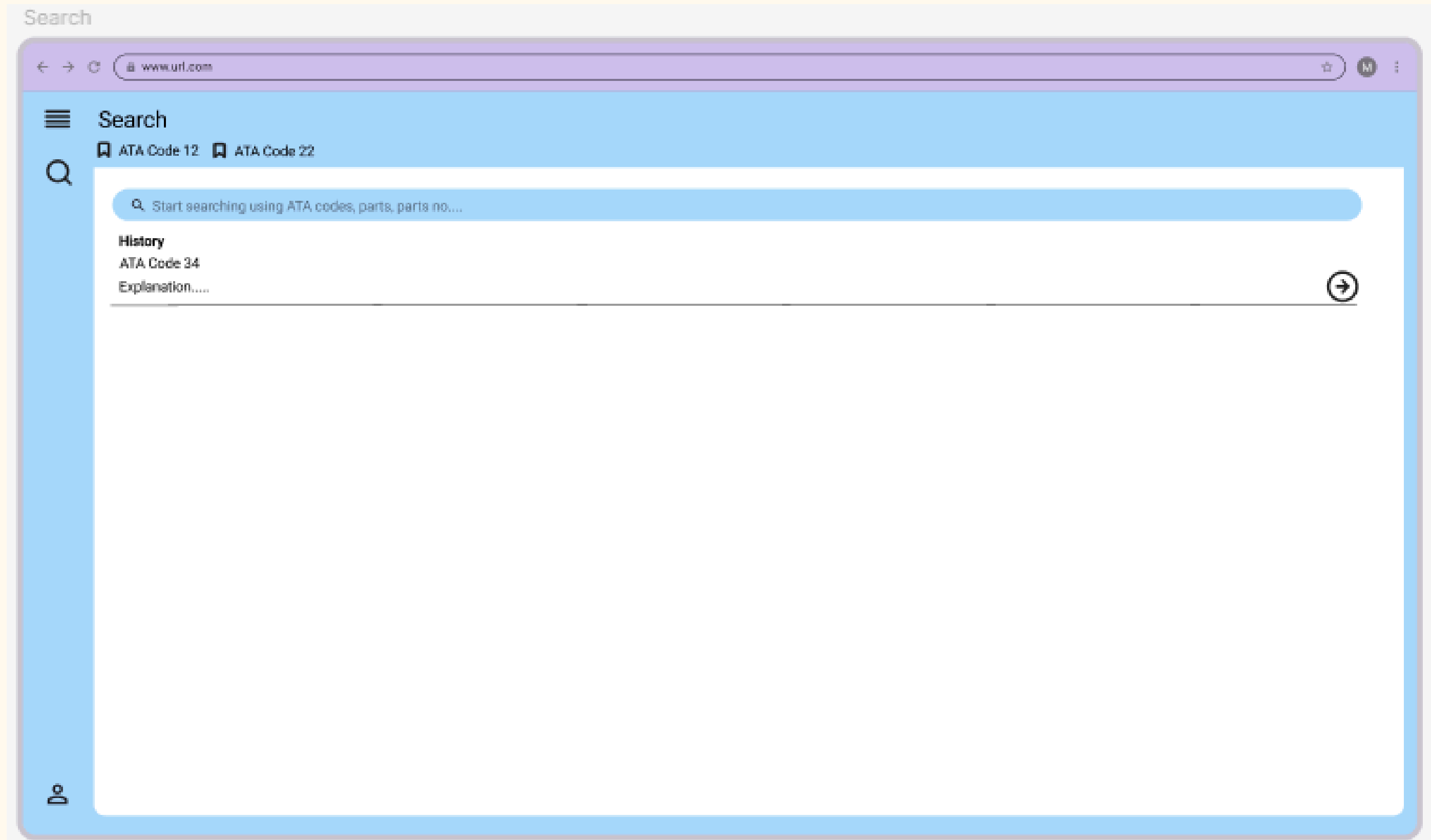
GANTT CHART

[illegible]

GANTT CHART

[illegible]

SKETCH



TECH STACK

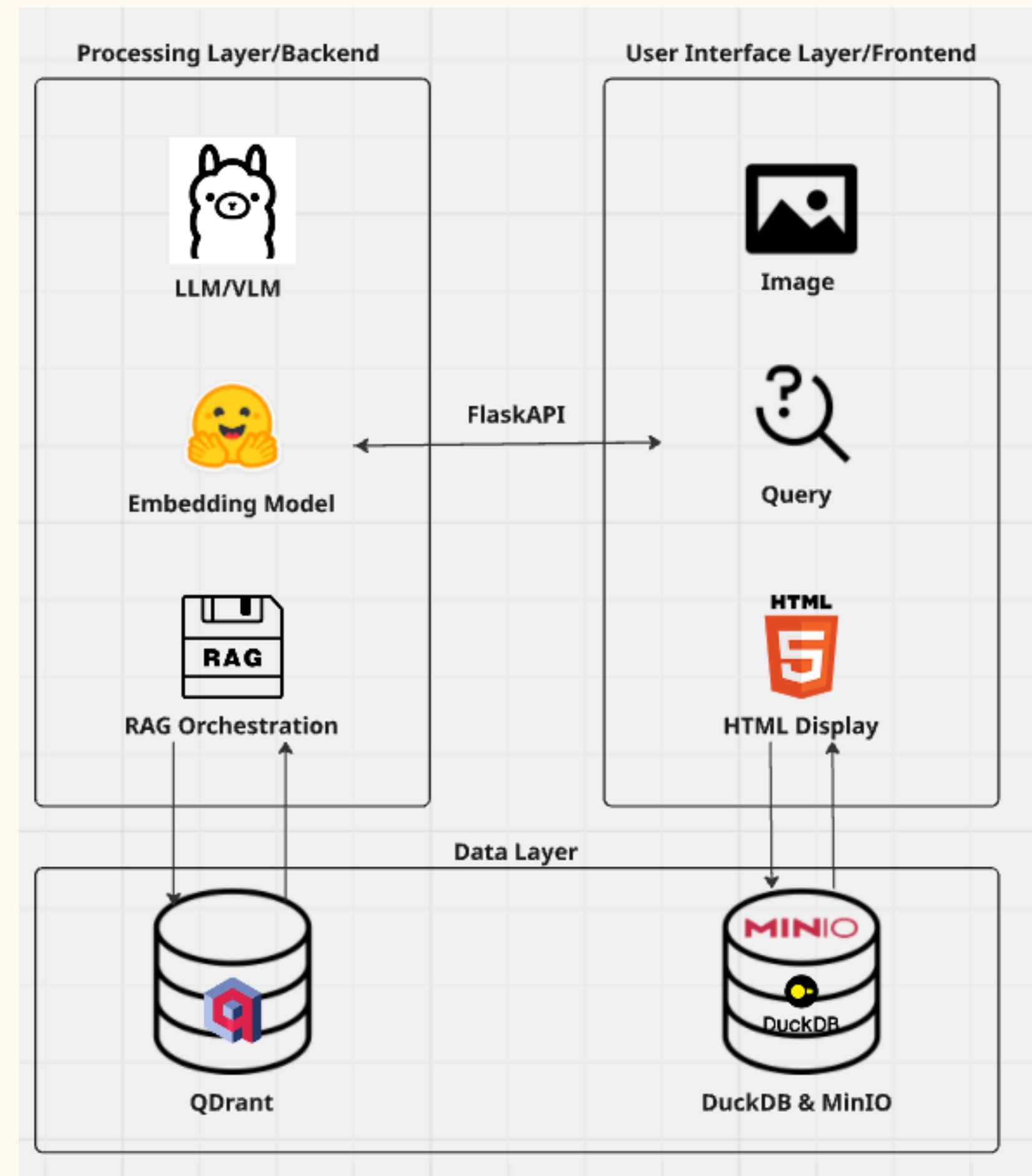


Purpose	Technology Used	What is it?	Why was it chosen?
Backend	Python	A versatile programming language with rich libraries for many functions.	Ideal for building robust pipelines, handling hierarchical data, and integrating third party software.
Frontend	HTML/CSS/JS	The standard trio for building interactive, user-friendly web interfaces.	
Embedding Model	all-MiniLM-L6-v2	A compact transformer model that converts text into dense vector embeddings.	It has the right balance between speed and accuracy. As well as efficiency that ensures smooth performance even on modest hardware.
LLM/VLM	phi4:14b/ qwen2.5-vl-7b	A language and vision-language model, optimized for local deployment with Ollama.	Phi-4 is a 14B parameter, state-of-the-art open model built upon a blend of synthetic datasets, data from filtered public domain websites, and acquired academic books and Q&A datasets.
Vector DB	QDrant	A vector database designed for semantic search and retrieval.	QDrant offers a higher performance which enables fast similarity search on embeddings, powering our RAG pipeline.
File DB (Blob Storage)	MinIO	An object storage system.	MinIO provides a scalable, self-hosted way to store large files like PDFs and integrates smoothly with our backend and keeps data under our control.
Website DB (Relational DB)	DuckDB	A database optimized for fast queries on structured data.	DuckDB is lightweight yet powerful, making it perfect for storing and querying structured metadata (like parsed codes, summaries, and user history).

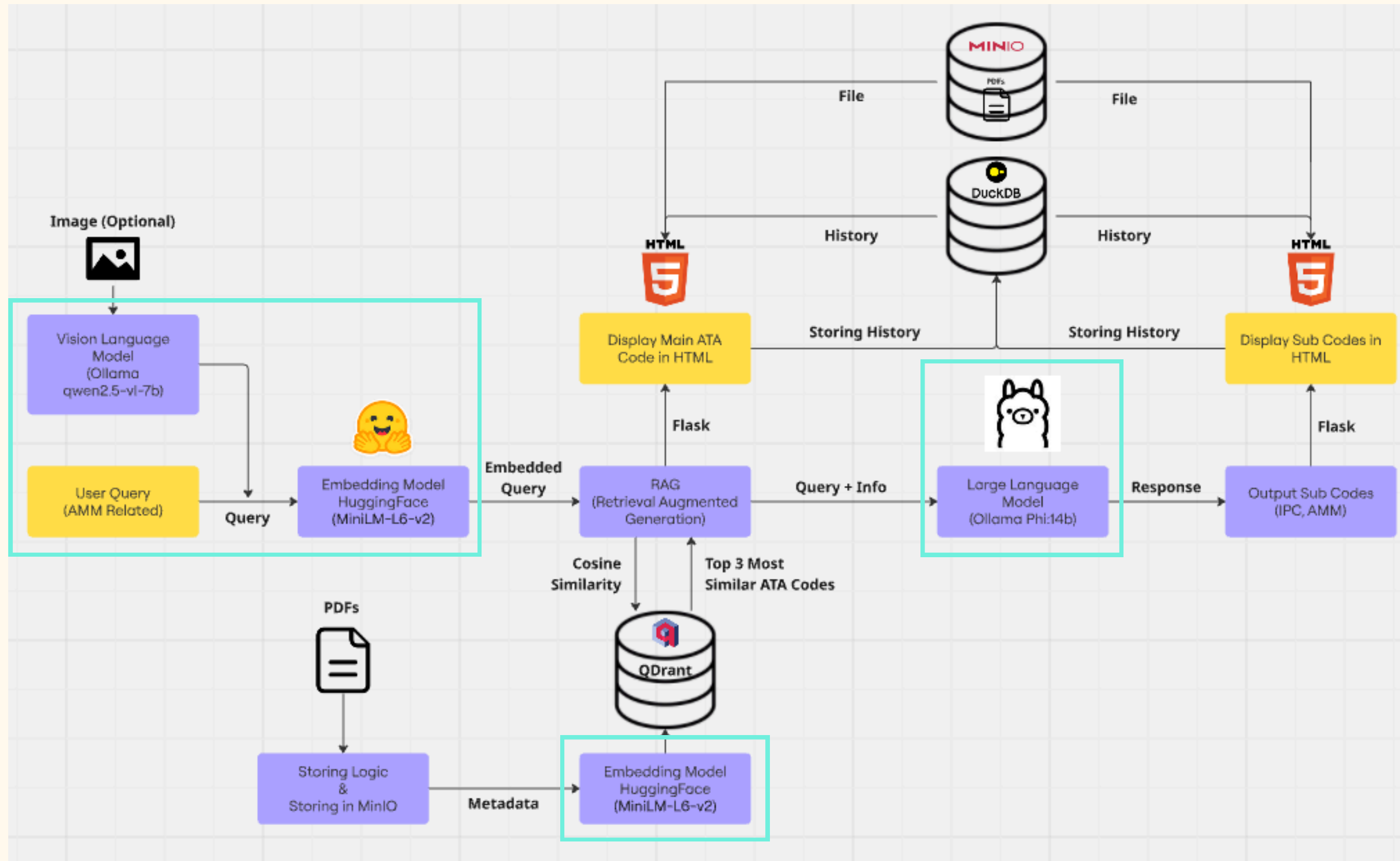


SYSTEM ARCHITECTURE

ALL LOCALLY HOSTED



DATA FLOW



IMPLEMENT

• LIST OF ACTIVITIES CARRIED OUT IN IMPLEMENT STAGE:

1. Research on **RAG Pipeline** and **LLM/VLM Capabilities**.
2. Design Web Application UI/UX and backend data flow.
3. Setting up Visual Studio Code, Ollama and Docker.
4. Experiment **RAG Pipeline** with different **embedding models**, **storing logic** and **LLM/VLM** for extraction of sub-codes and Image Search.
5. Choose the **best model and pipeline** with the highest accuracy.
6. Implement it into UI/UX on prototype web application.
7. Bug Fixes
8. Improvements on both Frontend and Backend based on user and industry feedback.
9. Demo and presentation of prototype.
10. Preparation of report and instruction manual.

```
Help
Internship-main
ChatBOT.py X ChatBOT2.py fileuploadstore.py search.html login.html uploadfile.html query.py Qdrant.py
Project > ChatBOT.py > get_main_codes
889 > def logout():...
887
888 @app.route('/upload', methods=['POST'])
889 > def upload_image():...
950
951 @app.route('/get_main_codes', methods=['POST'])
952 def get_main_codes():
953     is_direct_code_search = None
954     data = request.json
955     user_question = data.get('question', '')
956     worker_id = data.get('worker_id', '')
957
958     AMM_SRM_PATTERN = re.compile(r"\b(\d{2}-\d{2}-\d{2})\b")
959     IPC_PATTERN = re.compile(r"\b(\d{2}-\d{2}-\d{2}){([A-Z]?)\b}")
960     DOC_CODE_PATTERN = re.compile(r"\b(AMM|IPC|SRM)\b", re.IGNORECASE)
961     code_match = DOC_CODE_PATTERN.search(user_question)
962     ts = int(time.time())
963
964     if code_match:
965         code = code_match.group(1).upper()
966         if code == "AMM":
967             m = AMM_SRM_PATTERN.search(user_question)
968             if m:
969                 formatted_digits = m.group(1)
970                 results = handle_userinput_getdocuments(formatted_digits, 2) #AMM ONLY
971                 is_direct_code_search = len(results) == 1
972             else:
973                 striped = DOC_CODE_PATTERN.sub('', user_question).strip() #AMM ONLY WITH NLS
974                 results = handle_userinput_getdocuments(striped, 5)
975                 is_direct_code_search = 0
976         elif code == "SRM":
977             m = AMM_SRM_PATTERN.search(user_question)
978             formatted_digits = m.group(1)
979             results = handle_userinput_getdocuments(formatted_digits, 3) #SRM ONLY
980             is_direct_code_search = len(results) == 1
981         elif code == "IPC":
```

Containers [Give feedback](#)

Container CPU usage ⓘ
0.87% / 2800% (28 CPUs available)

Container memory usage ⓘ
2.56GB / 61.18GB

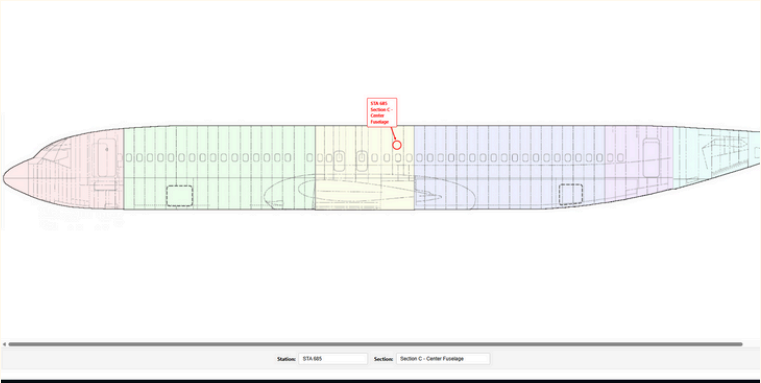
Show charts

Search

☰

Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Memor	Actions
☐	minionew	b3cd10cfec3e	minio/minio	9000:9000 ↗ Show all ports (2)	0.07%	88.89M	
☐	agitated_sander	8e6e46d64217	roboflow/rx	9001:9001 ↗	0.45%	2.32GB	
☐	brave_elion	d81e18dd6610	qdrant/qdr	6333:6333 ↗	0.4%	158.1M	



Audit Log

Time	Task Name	Task Description	Start
11/10/2024, 10:00:00 AM	Login	-	10:00:00
11/10/2024, 10:00:01 AM	Login	-	10:00:01
11/10/2024, 10:00:02 AM	Login	-	10:00:02
11/10/2024, 10:00:03 AM	Preload Search	Bootstrap Data	10:00:03
11/10/2024, 10:00:04 AM	Preload Search	Weather radar measurement	10:00:04
11/10/2024, 10:00:05 AM	Preload Search	Radome alert	10:00:05
11/10/2024, 10:00:06 AM	Preload Search	Ship or aircraft alert	10:00:06
11/10/2024, 10:00:07 AM	Preload Search	The integrating battery	10:00:07
11/10/2024, 10:00:08 AM	Preload Search	Weather check	10:00:08
11/10/2024, 10:00:09 AM	Preload Search	Location antenna	10:00:09
11/10/2024, 10:00:10 AM	Preload Search	GNIF antenna equipment	10:00:10
11/10/2024, 10:00:11 AM	Preload Search	GNIF antenna equipment	10:00:11
11/10/2024, 10:00:12 AM	Preload Search	GNIF antenna equipment	10:00:12
11/10/2024, 10:00:13 AM	Preload Search	Radome part disp	10:00:13
11/10/2024, 10:00:14 AM	Preload Search	Radome alert	10:00:14
11/10/2024, 10:00:15 AM	Preload Search	Capacitance Fuel probe test	10:00:15
11/10/2024, 10:00:16 AM	Preload Search	Fuel probe test	10:00:16
11/10/2024, 10:00:17 AM	Preload Search	Locality uncertainty	10:00:17
11/10/2024, 10:00:18 AM	Preload Search	GNIF Antenna uncertainty	10:00:18
11/10/2024, 10:00:19 AM	Login	-	10:00:19
11/10/2024, 10:00:20 AM	Search	Add Bootstrap Data	10:00:20
11/10/2024, 10:00:21 AM	Search	Forward Corner Luggage Report	10:00:21
11/10/2024, 10:00:22 AM	Search	10:00:22 (10:00:22)	10:00:22

IMPLEMENT

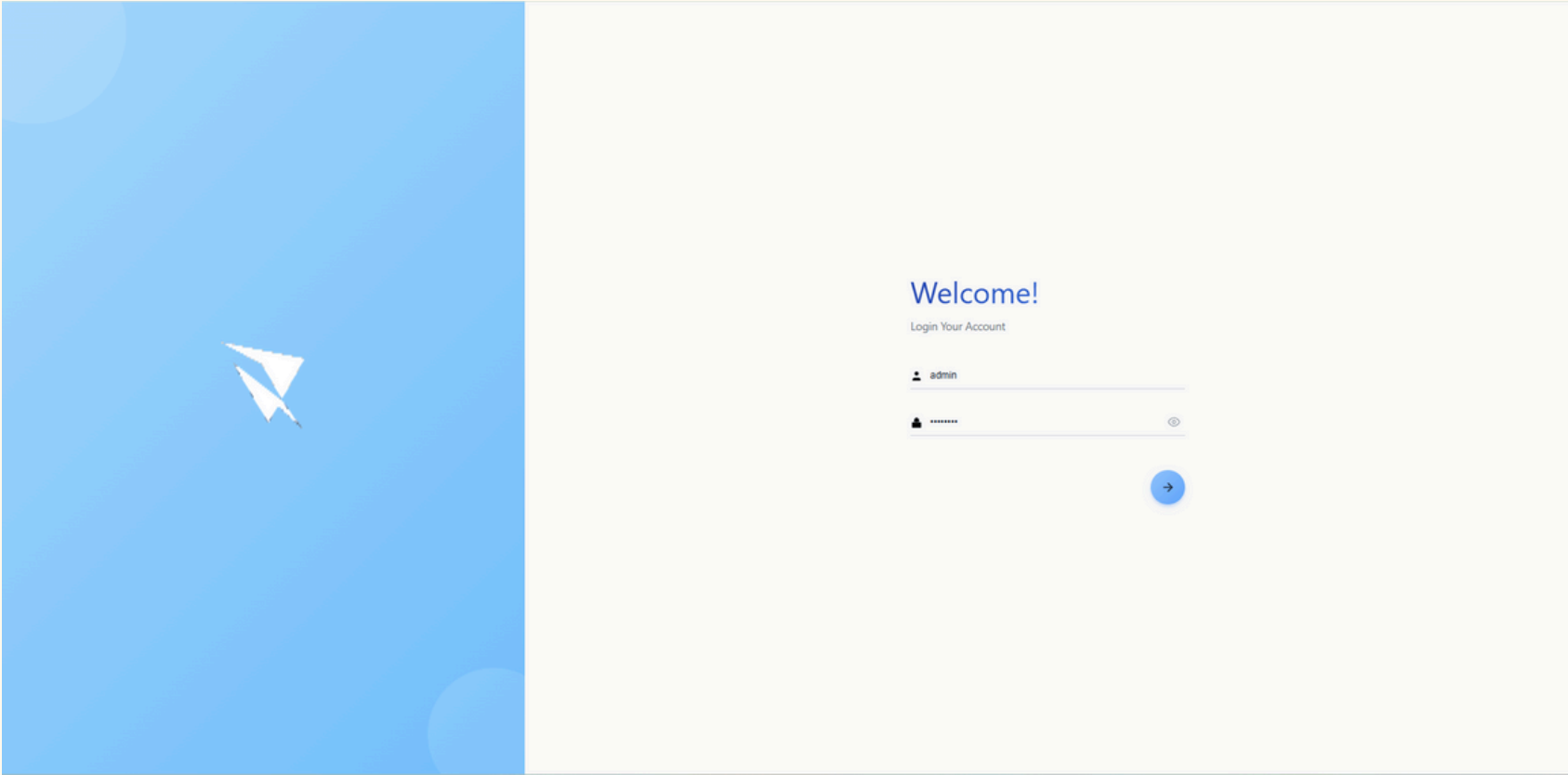
EXPERIMENTATION & INITIAL DECISION:

	Model 1	Model 2	Deciding Factor
Embedding Model	 bge-large-en	 all-MiniLM-L6-v2	Although bge-large-en has a higher accuracy, more suitable for natural language search, all-MiniLM-L6-v2 offers fast and great accuracy with keyword matching without relying on expensive hardware costs.
LLM/VLM	 Gemma3:12b	 llama3.1:8b	More accurate due to a larger size and multimodal, processing text and images and feature a 128K context window
Vector Database	 Chromadb	 Qdrant	Best for enterprise RAG, recommendation systems, multimodal search and it's more scalable than chromadb without relying on expensive hardware costs.

IMPLEMENT

FRONTEND:

Login Page



Search/Main Page & History Sidebar

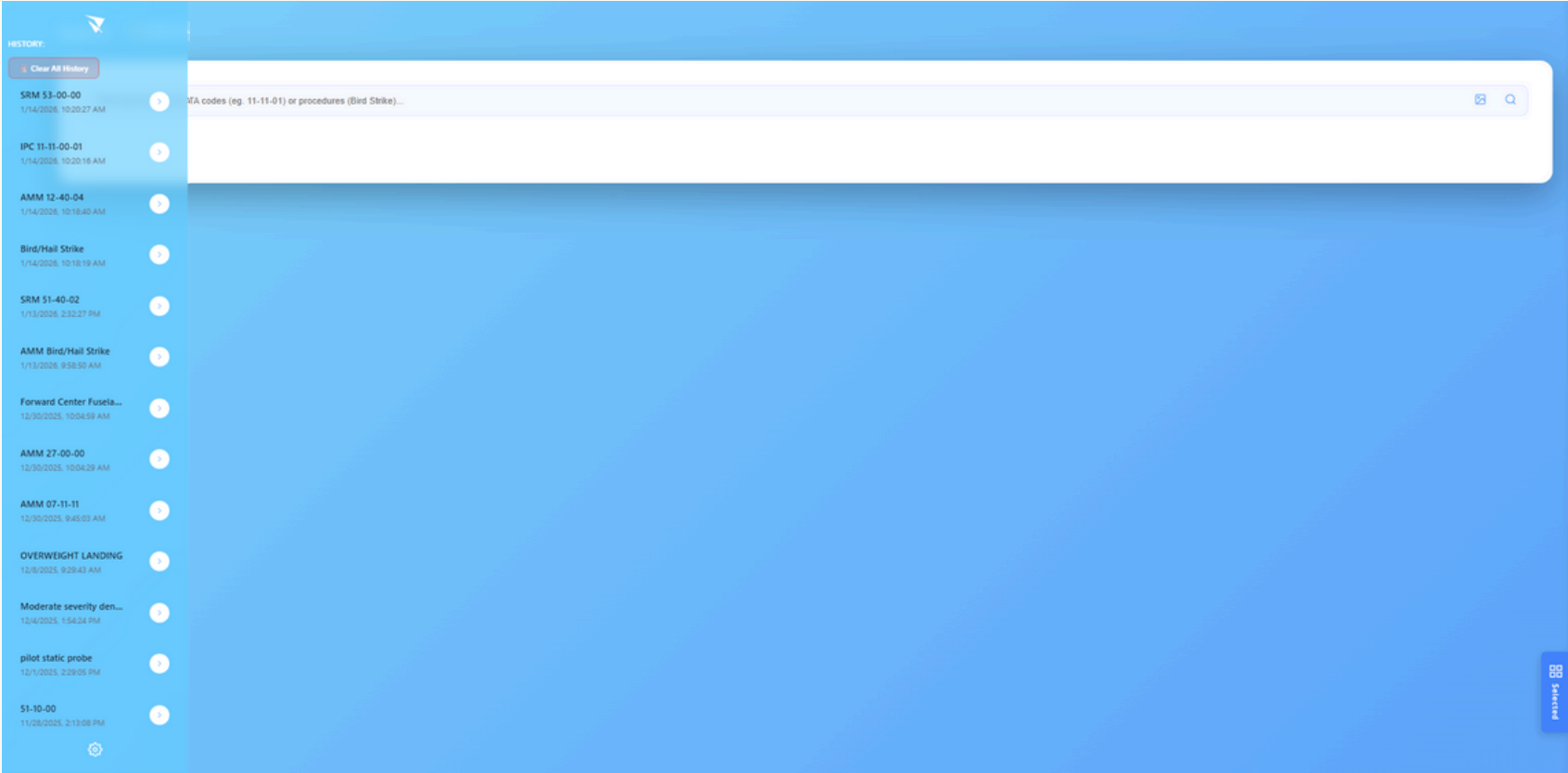
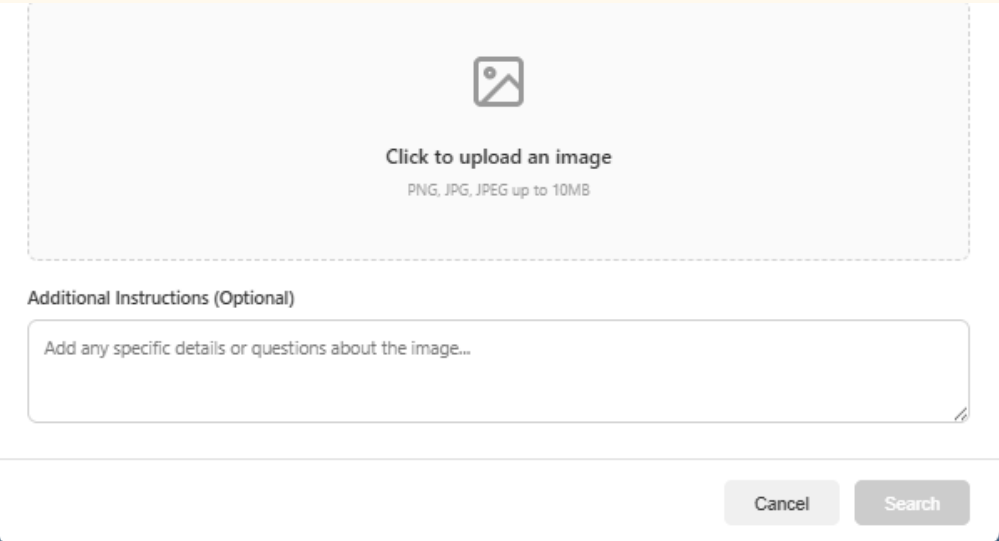


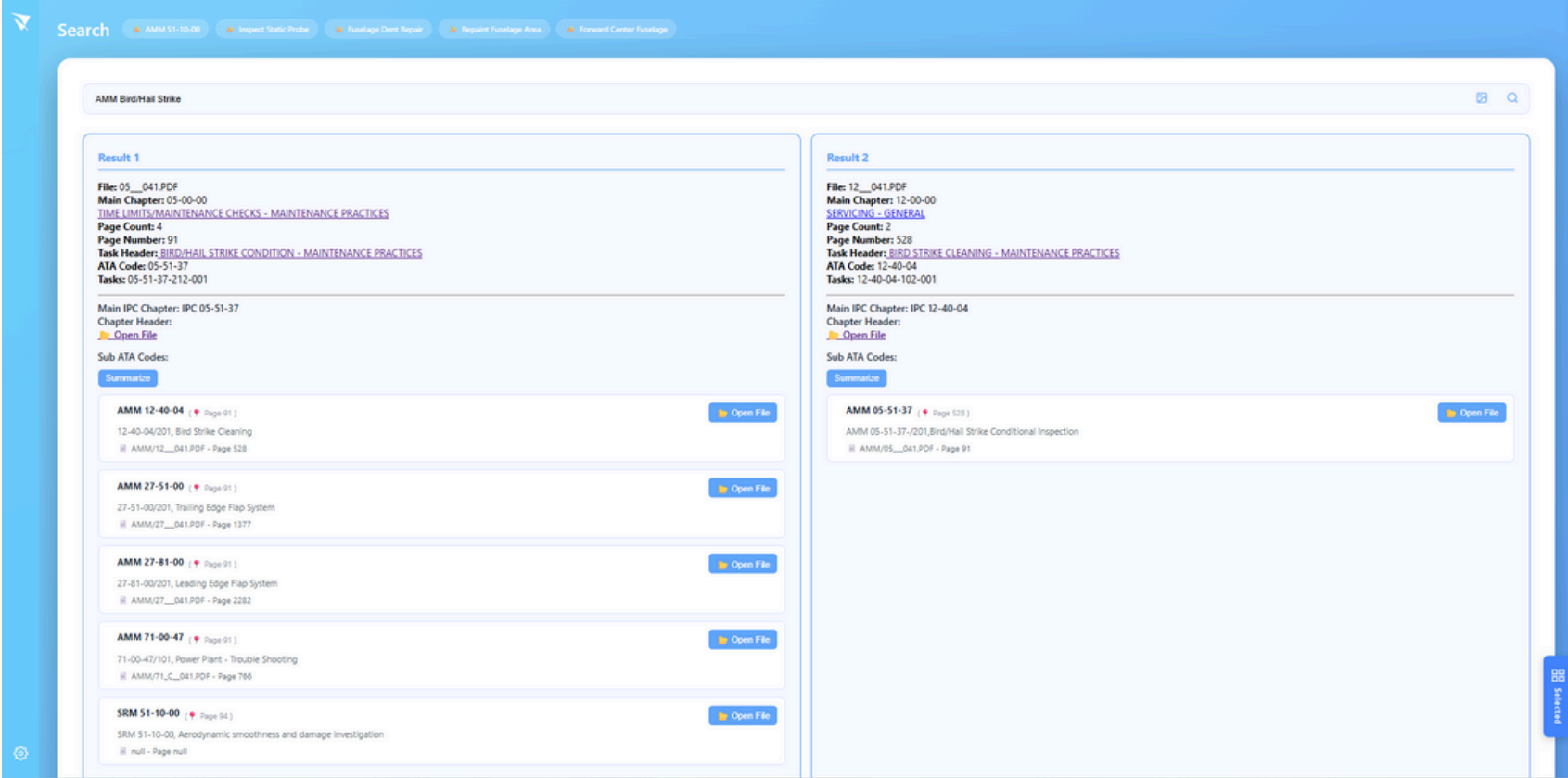
Image Search



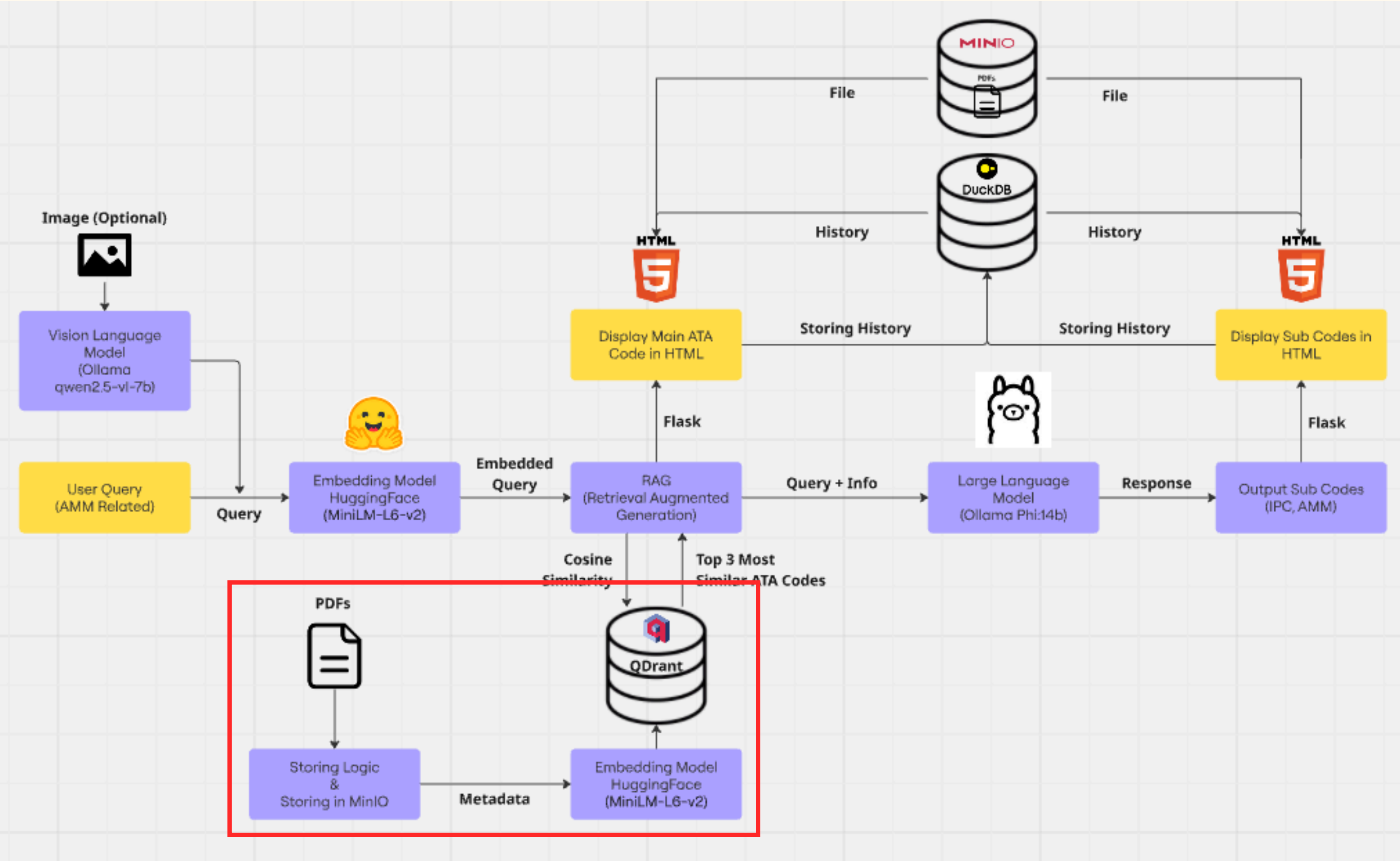
AI Suggestions (Based on user's history)



Manual and Code Extraction



IMPLEMENT



IMPLEMENT

BACKEND:

Pre-Processing & Storing:

- Information such as headers, ATA codes, and page numbers are extracted from the PDF using the PyPDF2 library and Regular Expression (regex).
- These elements are stored as metadata for future retrieval through AI, specifically embedding models, which capture semantic meaning and contextual relationships
- The metadata is inserted into the payload, with the corresponding header and chunk text tagged to it for organized storage and indexing.

```
# Page number and date
page_match = re.search(r'Page\s+(\d+)', text)
date_match = re.search(r'\b(?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)\s+\d{1,2}/\d{2}\b', text)

page_num = int(page_match.group(1)) if page_match else fallback_page_num
page_pdf = fallback_page_num

date = date_match.group(0) if date_match else None

return {
    "file_name": file_name,
    "file_path": object_name,
    "headers": header,
    "functions": json.dumps(functions) if functions else None,
    "sequential_numbers": json.dumps(sequential_numbers) if sequential_numbers else None,
    "tasks": json.dumps(tasks_id) if tasks_id else None,
    "task_codes": json.dumps(task_codes) if task_codes else None,
    "chapter": chapter,
    "section": section,
    "subject": subject,
    "ata_code": ata_code,
    "main_chapter": main_chapter_code,
    #manualpage
    "pagenumber": page_num,
    #pdfpage
    "pagepdf": page_pdf,
    "page_code": page_code,
    "page_code_count": final_count,
    "date": date,
    "first": is_first,
    "chunk_text": chunk_text,
    "indicator": indicator
}

points = []
if header:
    points.append(PointStruct(
        id=str(uuid4()),
        vector=embedding_function.encode(header),
        # vector=embedding_function.embed_query(header),
        payload={**metadata, "header": header}
    ))

for chunk in chunks:
    points.append(PointStruct(
        id=str(uuid4()),
        vector=embedding_function.encode([chunk])[0],
        # vector=embedding_function.embed_documents([chunk])[0],
        payload={**metadata, "chunk_text": chunk}
    ))

client.upsert(
    collection_name=COLLECTION_NAME,
    wait=True,
    points=points
)
```

EXAMPLE OF METADATA

TASK HEADER & CODE

HEADER

BODY TEXT

DATE, ATA CODE, PAGE NO.

FILE PATH & NAME,
PDF PAGE



737-300/400/500
MAINTENANCE MANUAL

TIME LIMITS/MAINTENANCE CHECKS – MAINTENANCE PRACTICES

TASK 05-00-00-912-001

1. Airworthiness Limitation Precautions

A. General

(1) Critical Design Configuration Control Limitations (CDCCLs)

(a) All occurrences of CDCCLs found in this chapter of the AMM are identified by this note after each applicable CDCCL design feature:

1) **NOTE:** CDCCL – Refer to the task: Airworthiness Precautions (AMM 5-00-00/201), for important information on Critical Design Configuration Control Limitations (CDCCLs).

(b) Design features that are CDCCLs are defined and controlled by Special Federal Aviation Regulation (SFAR) 88, and can be found in Airworthiness Limitations (AWL) and Certification Maintenance Requirements (CMR) document, D6-38278-CMR. CDCCLs are a means of identifying certain design configuration features intended to preclude a fuel tank ignition source for the operational life of the airplane. CDCCLs are mandatory and cannot be changed or deleted without the approval of the FAA office that is responsible for the airplane model Type Certificate, or applicable regulatory agency. A critical fuel tank ignition source prevention feature may exist in the fuel system and its related installation or in systems that, if a failure condition were to develop, could interact with the fuel system in such a way that an unsafe condition would develop without this limitation. Strict adherence to configuration, methods, techniques, and practices as prescribed is required to ensure the CDCCL is complied with. Any use of parts, methods, techniques or practices not contained in the applicable CDCCL must be approved by the FAA office that is responsible for the airplane model Type Certificate, or applicable regulatory agency.

(2) Airworthiness Limitation Instructions (ALIs)

(a) All occurrences of fuel tank system ALIs found in this chapter of the AMM are identified by this step after the General section in the applicable ALI inspection task:

1) ALI – Refer to the task: Airworthiness Limitation Precautions (AMM 5-00-00/201), for important information on airworthiness limitation instructions (ALIs).

EFFECTIVITY

ALL

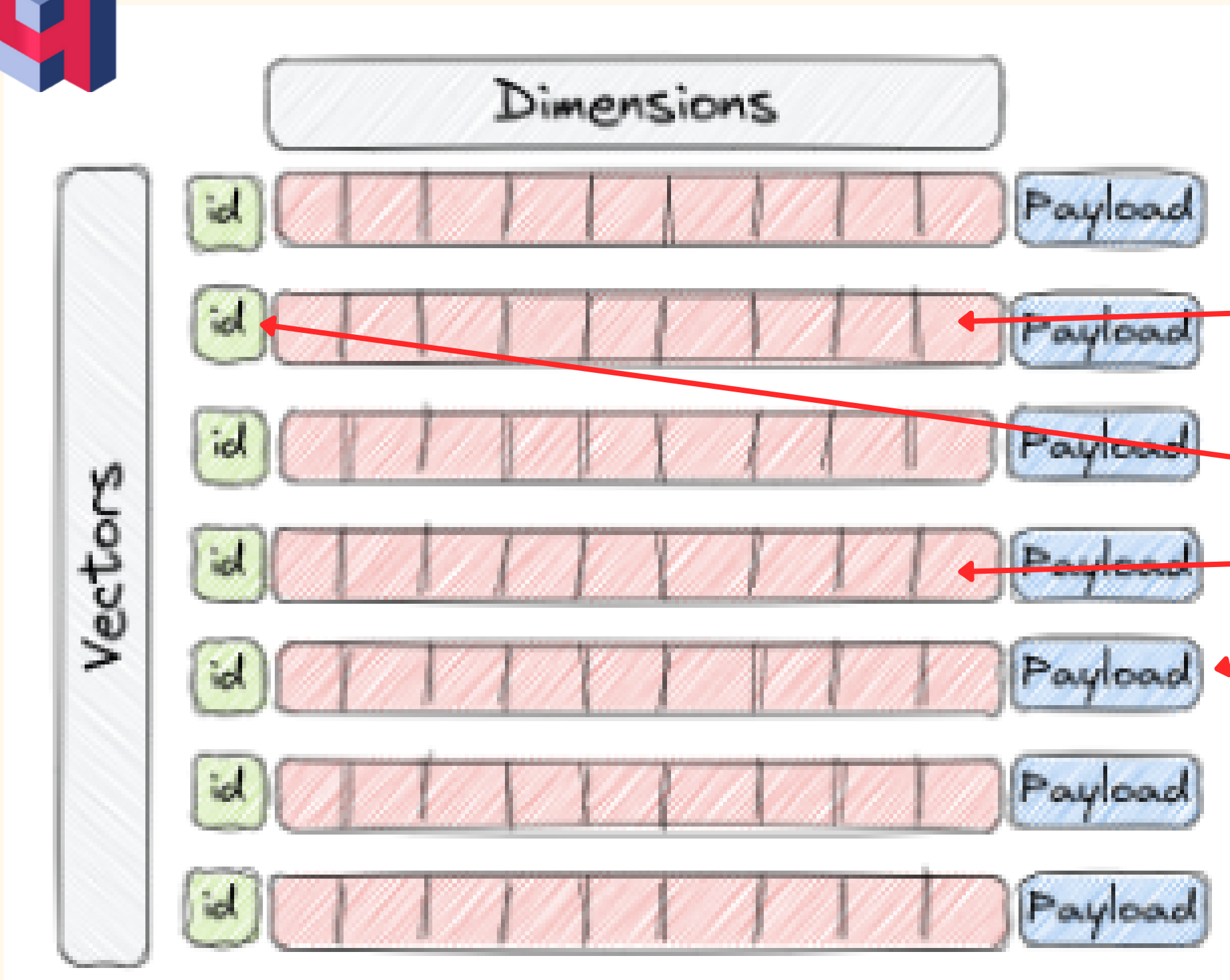
01

05-00-00

Page 201
Jan 12/06

BOEING PROPRIETARY – Copyright (C) – Unpublished Work – See title page for details.

EXAMPLE OF STORING

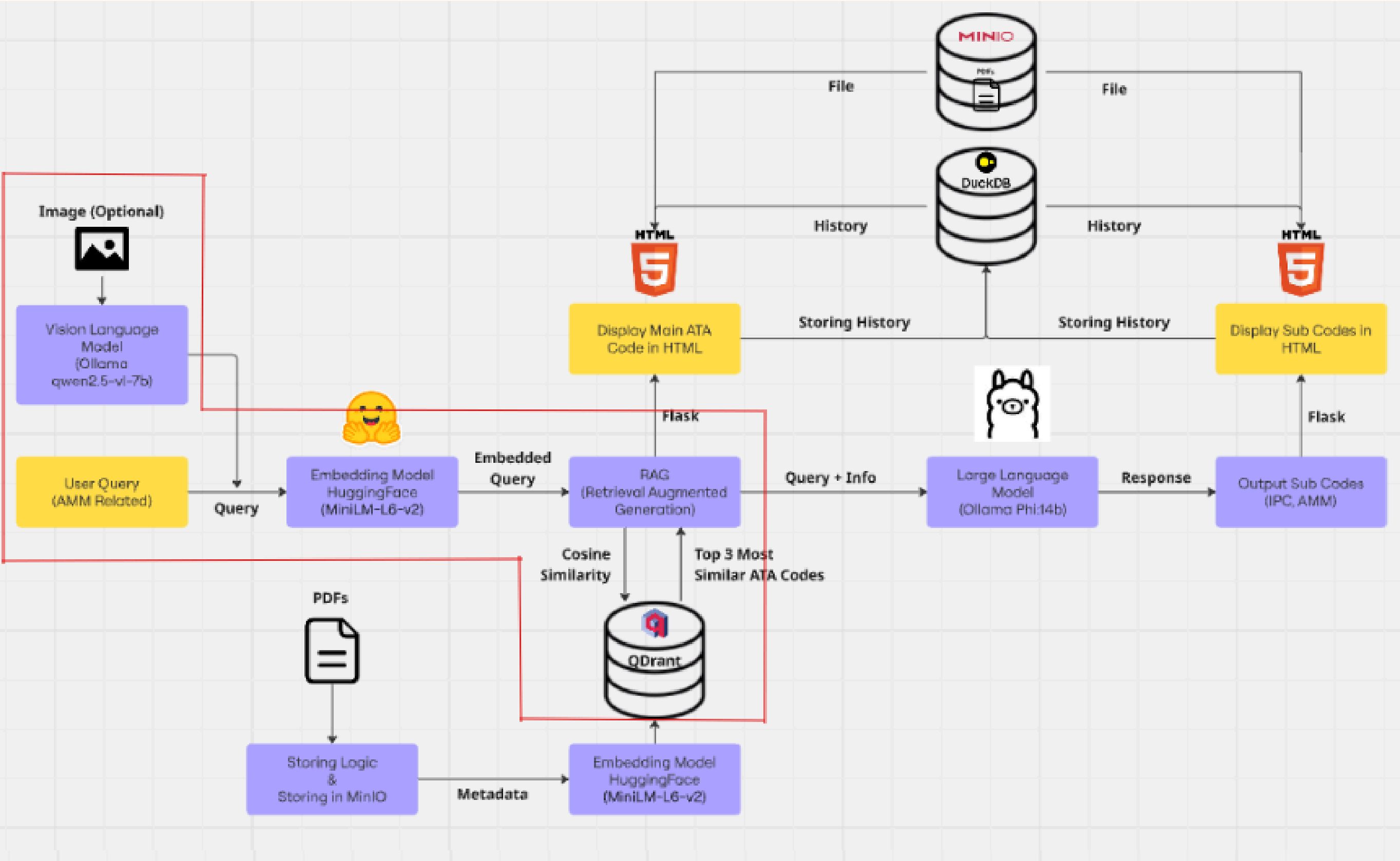


```
points = []
if header:
    points.append(PointStruct(
        id=str(uuid4()),
        vector=embedding_function.encode(header),
        payload=metadata
    ))

for chunk in chunks:
    points.append(PointStruct(
        id=str(uuid4()),
        vector=embedding_function.encode([chunk])[0],
        payload=metadata
    ))

client.upsert(
    collection_name=COLLECTION_NAME,
    wait=True,
    points=points
)
```

IMPLEMENT



IMPLEMENT

BACKEND:

Retrieving Logic (Semantic Search):

- The system retrieves the **top-K** results along with their associated **metadata** using **cosine similarity**. These results are then streamed to the frontend for display, enabling users to view both the retrieved content and its contextual information in real time.

Interaction with Frontend:

- In this example, the backend streams progress updates and extracted sub-codes to the frontend in real time through **FlaskAPI**. As soon as codes are retrieved, they are sent immediately, allowing the frontend to display search progress alongside the extracted results.

```
AMM_first_result = client.query_points(
    collection_name=COLLECTION_NAME,
    query=query_vector,
    limit=3,
    with_payload=True,
    score_threshold=0.2,
    query_filter=Filter(
        must=[
            FieldCondition(key="indicator", match=MatchValue(value="AMM")),
            FieldCondition(key="first", match=MatchValue(value=1))
        ])
    )

for point in AMM_first_result.points:
    responsetoquestion = ""
    page_chunks = []
    first_payload = point.payload or {}
    page_code = first_payload.get("page_code", 'Unknown')

    if page_code in seen_page_codes:
        continue
    seen_page_codes.add(page_code)

    newfilename = first_payload.get('file_name', 'Unknown')
    newpageNumber = first_payload.get('pagenumber', 'Unknown')
    filepath = first_payload.get('file_path', 'Unknown')
    header = first_payload.get('headers', 'Unknown')
    tasks = first_payload.get('task_codes', 'Unknown')
    chunk_text = first_payload.get("chunk_text", 'Unknown')
    first_page_pdf = first_payload.get("pagepdf", "Unknown")
    final_page_count = first_payload.get("page_code_count", 'Unknown')
    main_chapter = first_payload.get("main_chapter", "Unknown")
```

```
@app.route('/generate_subcodes', methods=['POST'])
def chat():
    data = request.get_json()
    user_question = data.get("question", "")
    selected_codes = data.get('selected_codes', [])

    if not user_question:
        return jsonify({"error": "No question provided"}), 400

    def generate():
        start_time = time.time()

        # Step 1: Search for documents (10%)
        yield send_progress(10, "Searching documents...")

        match = re.search(r"\b\d{2}-\d{2}-\d{2}\b", user_question)

        if match:
            formatted_digits = match.group(0)
            results = handle_userinput_getdocuments(formatted_digits, 2)
        else:
            results = handle_userinput_getdocuments(user_question, 1)

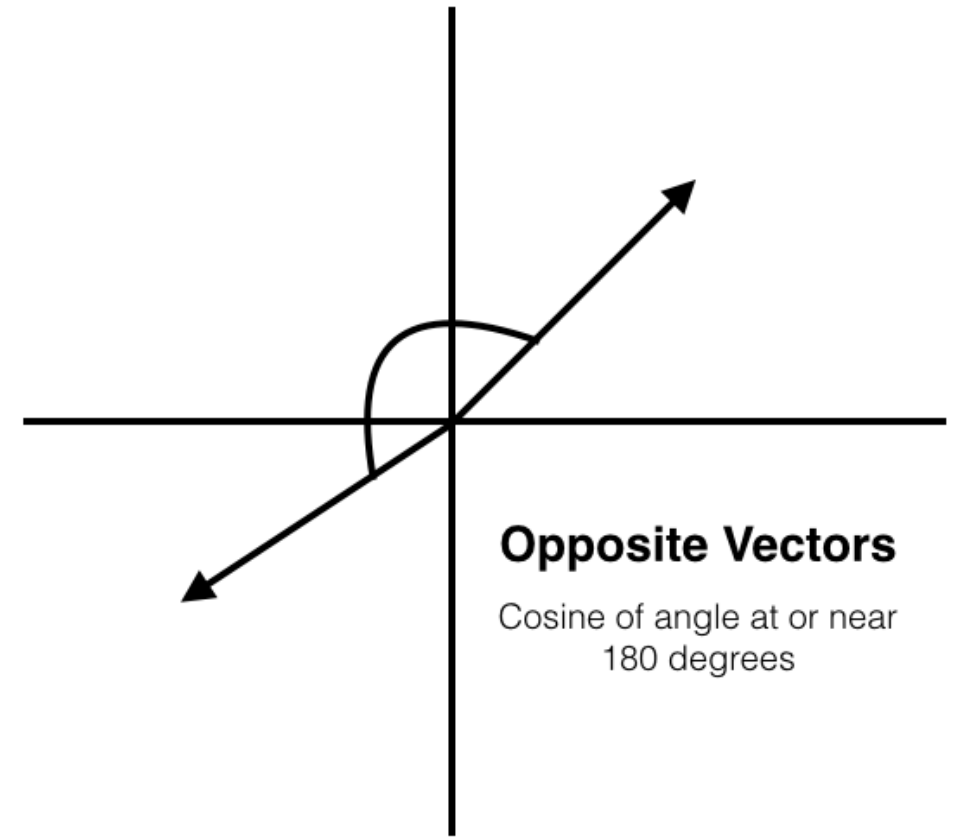
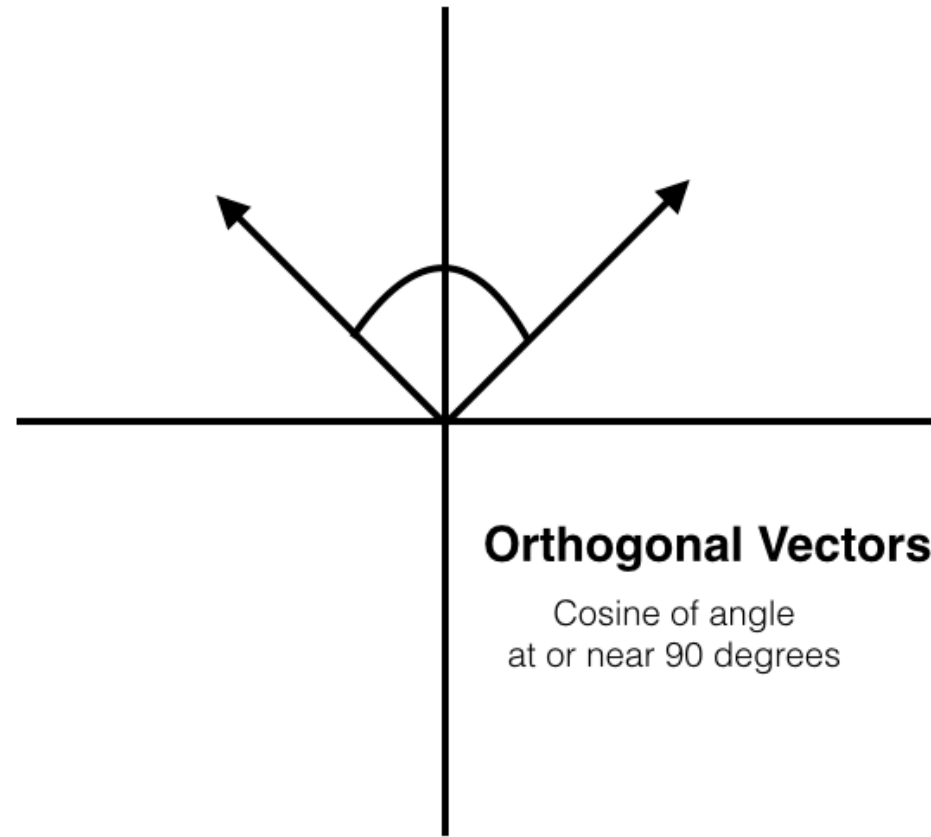
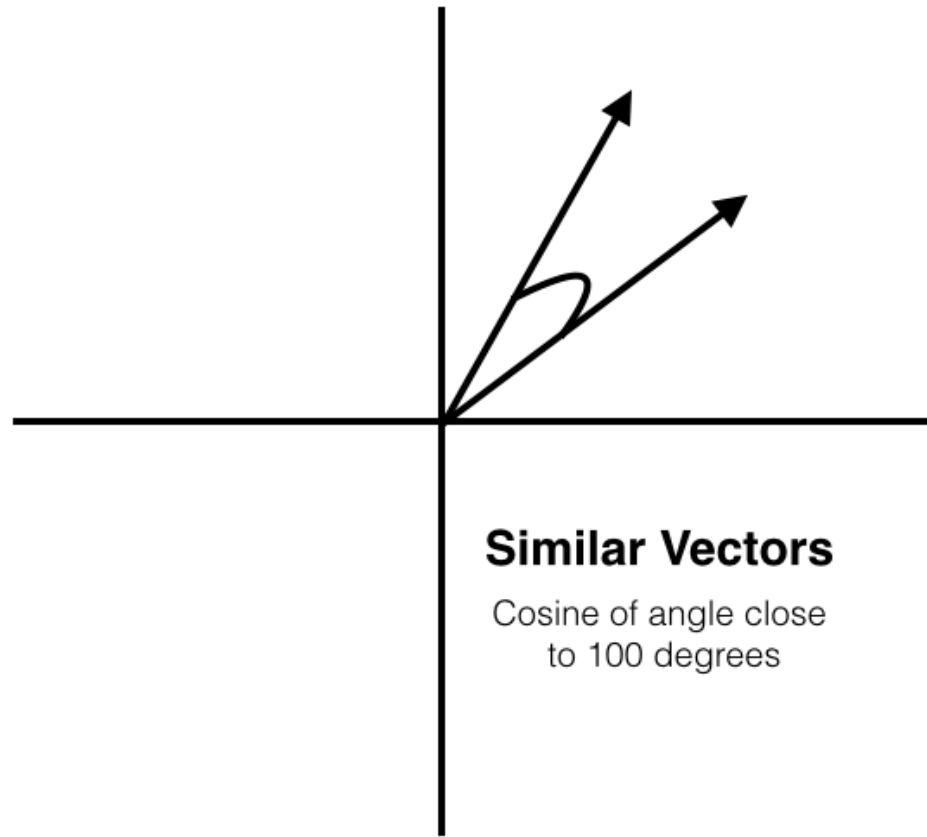
        filtered_results = [r for r in results if r['main_ata_code'] in selected_codes]

        if not filtered_results:
            yield f"data: {json.dumps({'error': 'No results found'})}\n\n"
            return

        yield send_progress(20, f"Found {len(filtered_results)} AMM document(s)")

    all_responses = []
    base_progress = 20
    progress_per_result = 60 / len(filtered_results)
```

COSINE SIMILARITY



VLM PROMPT

```
prompt = f"""You're an aircraft surface maintenance expert for Boeing 737-300/400/500.
```

```
Context: {extra_prompt}
```

```
Analyze the image and use the information if given above to describe the type and severity of the defect in terms of low, moderate, high.
```

```
If possible, also include the location of the defects.
```

```
In your response, include the defect, its location and the severity ONLY.
```

```
Example responses:
```

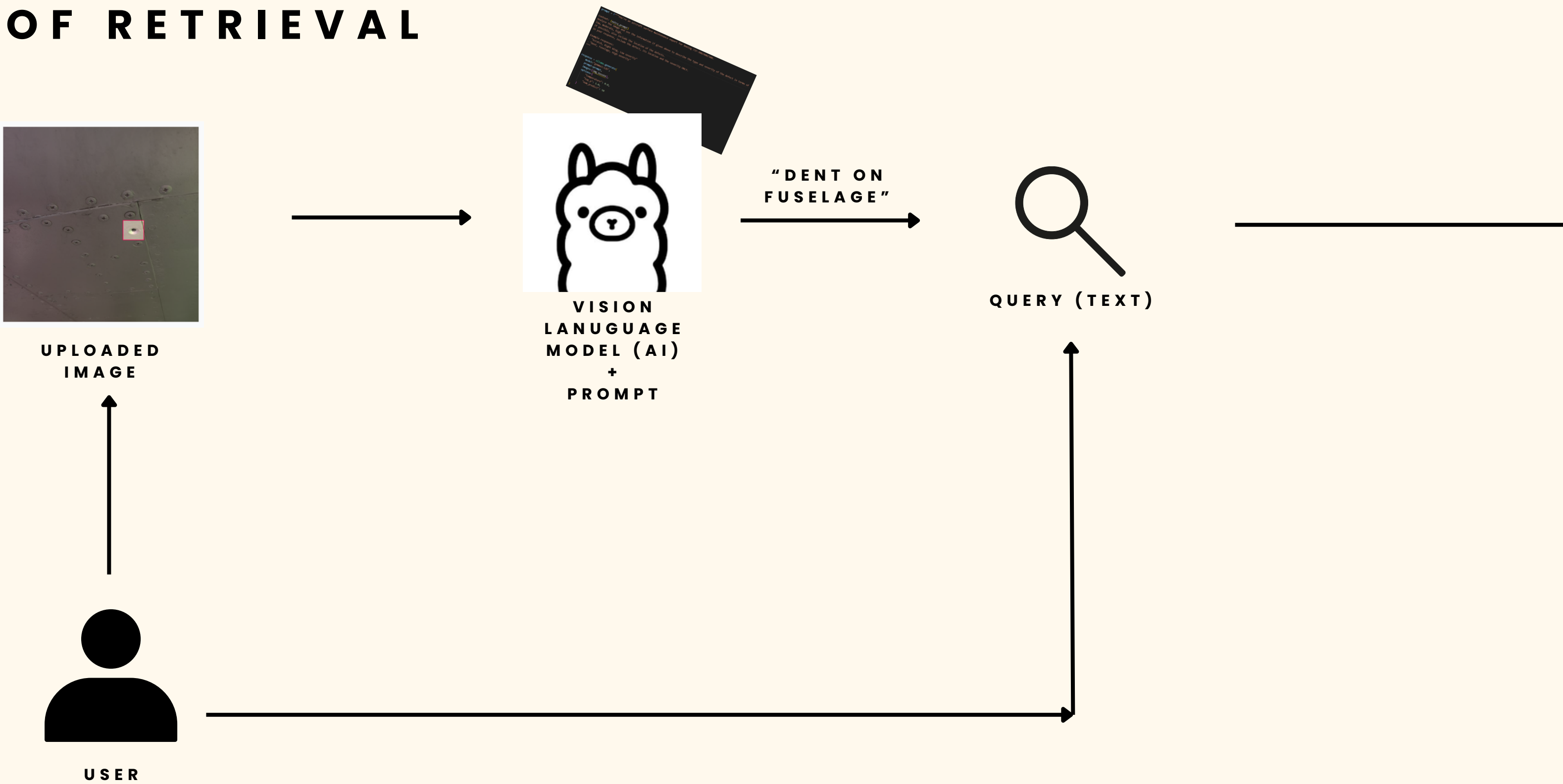
```
- "Scratch, Right Wing, Low severity"
```

```
- "Dent, Fuselage, High severity"
```

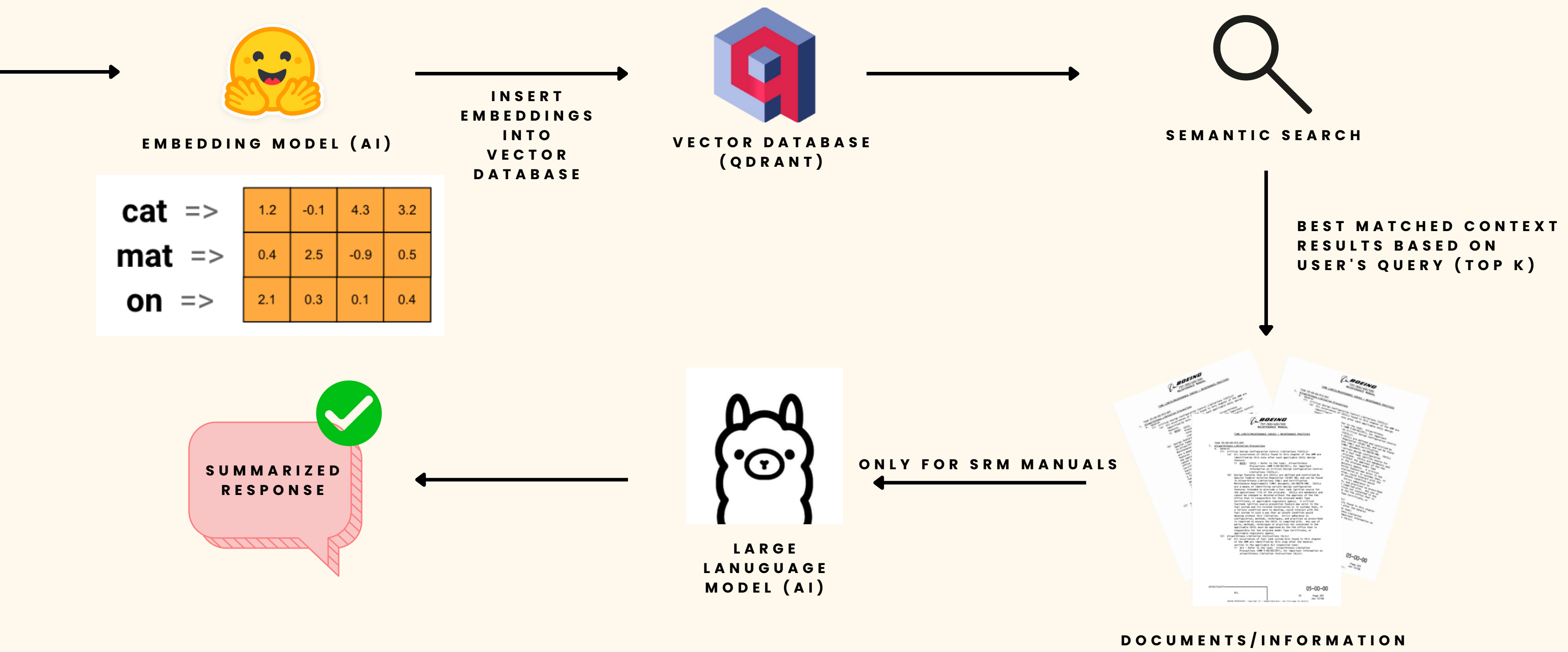
```
"""
```

```
response = ollama.generate(  
    model="gemma3:12b",  
    prompt=prompt,  
    images=[img_base64],  
    options={  
        "temperature": 0.0,  
        "top_p": 1.0,  
        "num_predict": 50  
    }  
)
```

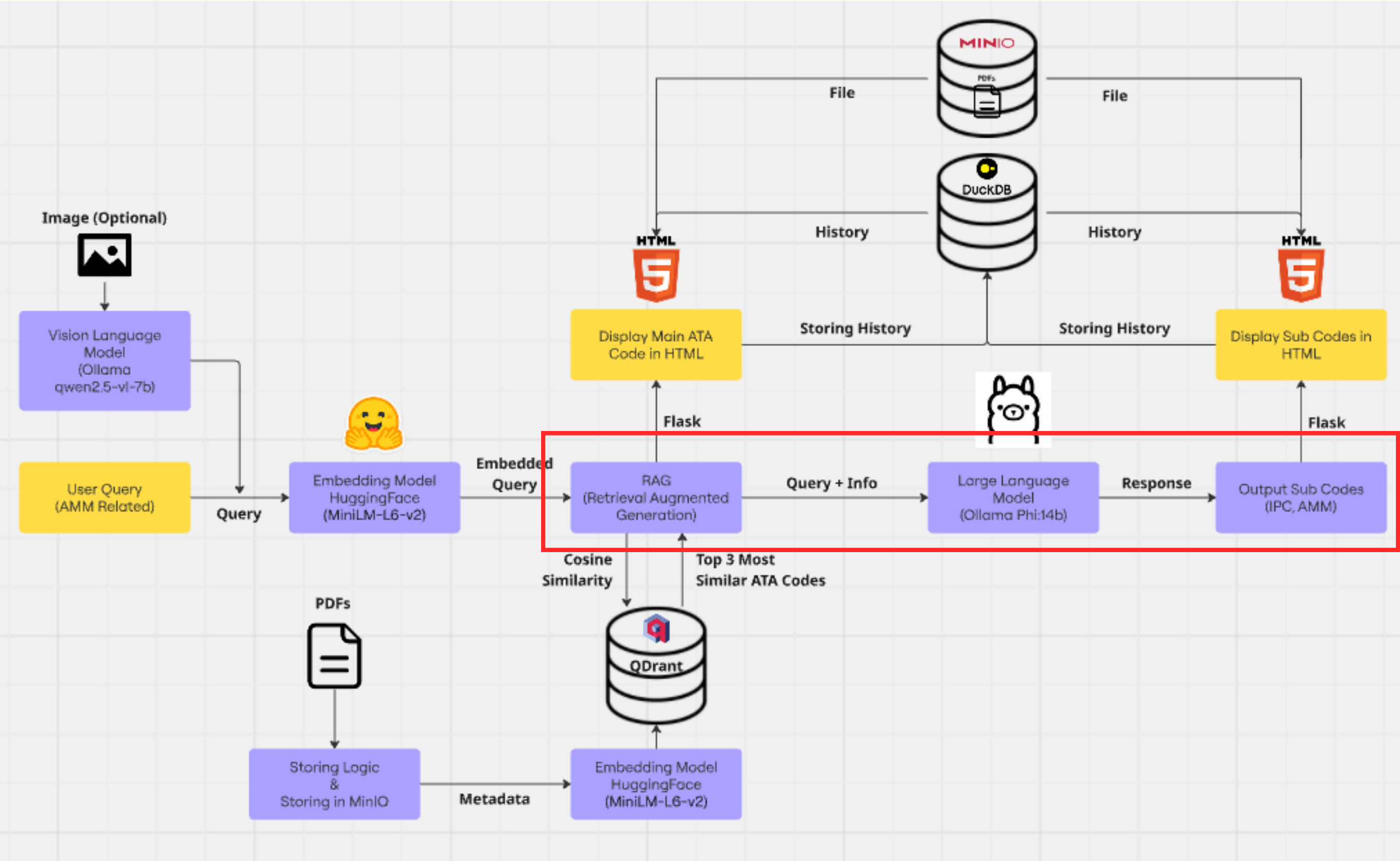

EXAMPLE OF RETRIEVAL



EXAMPLE OF RETRIEVAL



IMPLEMENT



IMPLEMENT

BACKEND:

Code Extraction Logic:

- By applying the correct prompts, a Large Language Model (LLM) can process the body text of each selected page and extract the corresponding sub-codes.

Parameters:

- Temperature: 0.0 (Ensures no randomness)
- Top_P: 0.9 (Provide additional contextual description for each ATA code)
- Num_predict: 500 (To support multiple codes extraction)

```
def extract_codes_from_single_page(page_text, main_ata_code, page_number):
    prompt = f"""
    You are an assistant that extracts maintenance codes from aircraft maintenance manuals.

    Text from page {page_number}:
    {page_text}

    Extract all the codes under the "References" section.

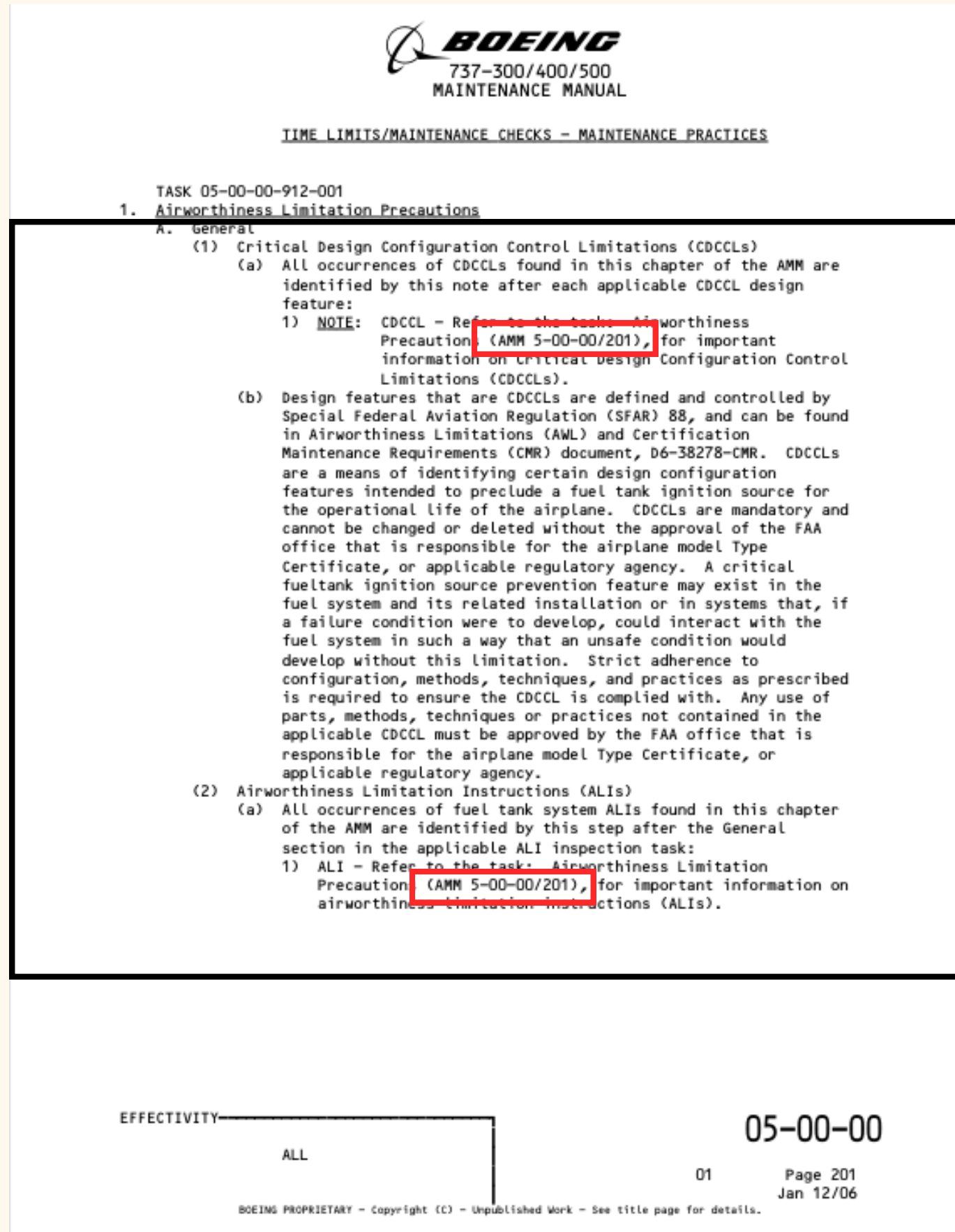
    Extract ALL codes ONLY in this format:
    - ATA Codes: AMM/SRM DD-DD-DD (e.g. AMM 05-41-00, SRM 32-21-01, 46-02-10)
    - IPC Codes: IPC DD-DD-DD(-DDD) (e.g. IPC 34-05-12, IPC 11-11-00-37B, IPC 11-11-00-37)
    EXCLUDE:
    - {main_ata_code}
    - Steps, Tasks (e.g. S 862-070, S 062-075)

    For each code found, provide brief description (max 8 words)

    Respond with ONLY a valid JSON array of strings. No markdown, no explanations.
    This is an example only, DO NOT INCLUDE THESE in your response: ["12-40-04/201,Bird Strike Cleaning, SRM 27-81-11,Leading Edge Flaps, AMM 32-45-00/601,Tires and Wheels
    """

    try:
        response = ollama.generate(
            model=OLLAMA_MODEL,
            prompt=prompt,
            options={
                "temperature": 0.0,
                "top_p": 0.9,
                "num_predict": 500
            }
        )
```

EXAMPLE OF EXTRACTING



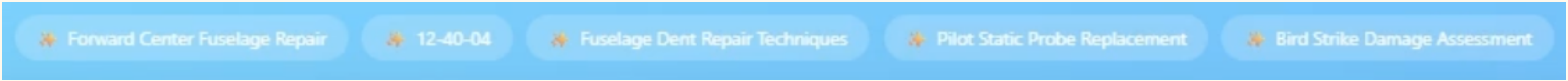
LARGE LANGUAGE MODEL (AI)
+
PROMPT

AMM 5-00-00

RESPONSE

IMPLEMENT

BACKEND:



AI Suggestions:

- The prompt is designed to generate query recommendations by leveraging the current user’s history as contextual guidance. The output is constrained to between 2 and 4 words per suggestion, with a requirement to produce 3 to 8 recommendations in total. Additionally, if relevant, one of the suggestions must include an ATA code.

```
prompt = f"""
You are an aviation maintenance assistant. Based on these recent ATA code searches: "${historyContext}"

Generate 3-8 SHORT related search suggestions for aircraft systems, ATA codes, maintenance, or parts. Each suggestion should be 2-4 words maximum.
For ATA codes, use the format "XX-XX-XX" (e.g. 10-11-20) AND include at least one code suggestion if relevant.

Rules:
- Only output the suggestions, one per line
- No numbering, bullets, or explanations
- Focus on related aircraft systems or maintenance tasks
- Keep suggestions SHORT and specific
- Use aviation/ATA terminology from AMM manuals

Examples of good suggestions:
Engine Oil System
10-11-20
Fuel Pump Check
Bird Strike Inspection

Your suggestions:
"""
```

IMPLEMENT

BUG FIXING:

- File Location not working after network change
- Selected codes resetting after refresh of page
- Decrease area or fix regex to increase accuracy of finding headers
- History Ranking
- Capture tasks header more accurately
- History limit and delete
- Deleting & Recreating of Vector and Blob Database
- Fix Summarising of Codes

IMPROVEMENTS TO BE MADE AND FEEDBACK:

- SRM Integration
- Severity and Location of Defects for Damage Assessment
- Audit Logging
- Different Machine Accessibility
- Decrease in Search time (if possible)

IMPLEMENT

IMPROVEMENTS TO BE MADE AND FEEDBACK:

- SRM Integration/AI Summarisation
- Audit Log
- Multiple Machine Accessibility

Found 3 SRM Document(s)

These SRM documents are displayed for reference:

51-50-02

File: 51__053.PDF

Main Chapter: 51-00-00

[GENERAL - STRUCTURES](#)

Page Count: 58

Page Number: 909

Task Header: [GENERAL - SUPPORT OF THE AIRPLANE FOR REPAIR](#)

ATA Code: 51-50-02

Summary ("Refer to manual for accurate instructions"): This text outlines procedures for supporting an airplane in a jig position before major structural repairs to the fuselage, stabilizers, or wings. It references specific documents and diagrams for support procedures and equipment, noting that recovery of damaged airplanes requires separate documentation. If standard support methods are inadequate, contact The Boeing Company is advised.

51-70-11

File: 51__053.PDF

Main Chapter: 51-00-00

[GENERAL - STRUCTURES](#)

Page Count: 21

Page Number: 1661

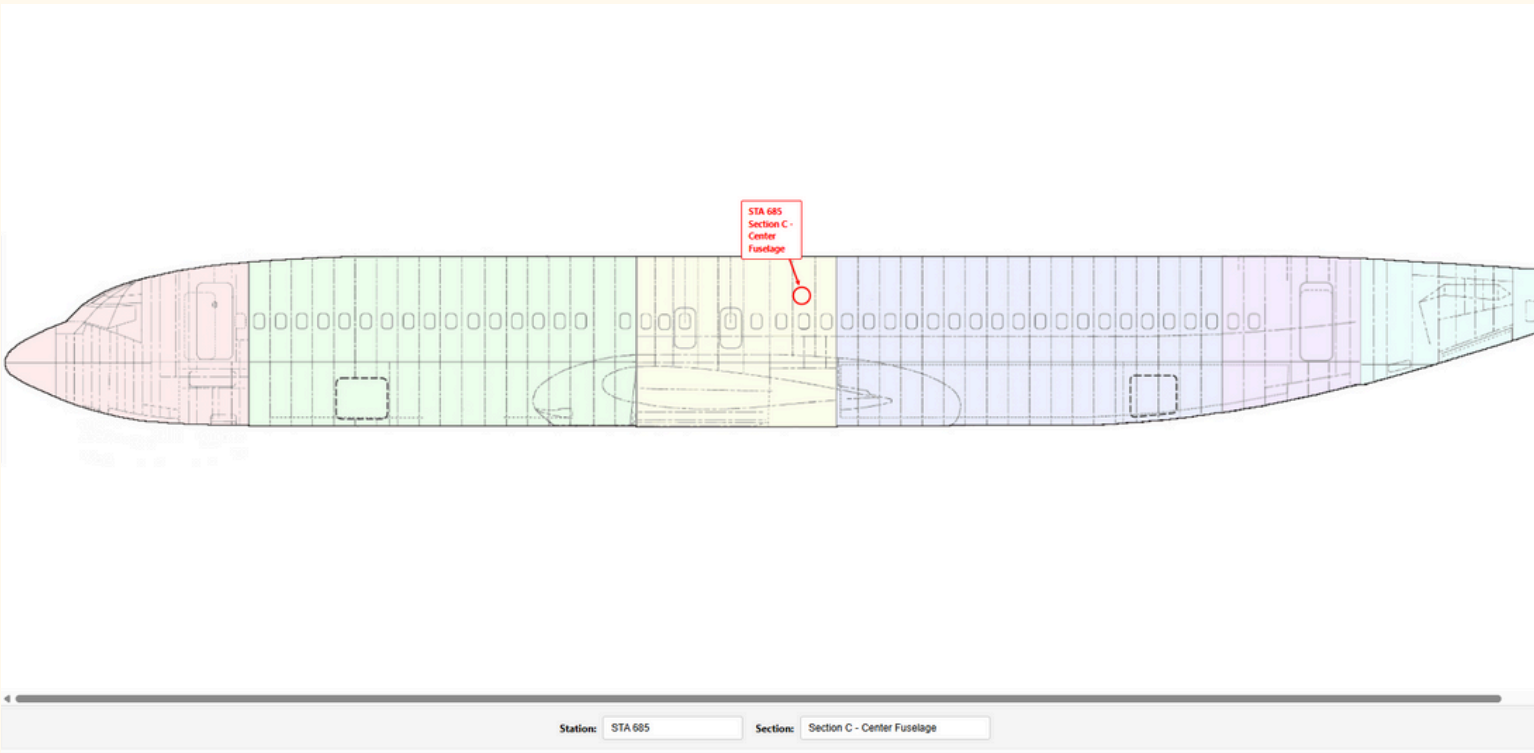
Task Header: [REPAIR GENERAL - TYPICAL FORMED SECTION REPAIR](#)

ATA Code: 51-70-11

Summary ("Refer to manual for accurate instructions"): This text outlines general guidelines for repairing formed sections on airplanes, specifically those with Service Bulletin 737-21-1149. It specifies conditions under which repairs are approved near stabilizer beams or existing repairs, emphasizing the need for supplemental inspections and referencing specific Boeing documents for critical design limitations and damage tolerance evaluations. For wing structures, it highlights requirements for fastener protection against lightning and electrical faults. Additionally, if a repair is on Fatigue Critical Baseline Structure, a Damage Tolerance Evaluation is necessary within 24 months of service return.

Audit Log		
Time	Task Name	Task Description
1/16/2026, 10:00:58 AM	Login	-
1/14/2026, 10:22:59 AM	Login	-
1/14/2026, 10:22:06 AM	Logout	-
1/14/2026, 10:21:06 AM	Preview Search	Perform INSPECTION AND REMOVAL OF DAMAGE, Crack, Section C - Center Fuselage, STA 685, High severity
1/14/2026, 10:21:06 AM	Image Search	-
1/14/2026, 10:20:27 AM	Search	SRM 53-00-00
1/14/2026, 10:20:16 AM	Search	IPC 11-11-00-01
1/14/2026, 10:19:46 AM	Preview Search	AMM 34-11-01
1/14/2026, 10:19:26 AM	Preview Search	Inspect Static Probes
1/14/2026, 10:18:40 AM	Search	AMM 12-40-04
1/14/2026, 10:18:19 AM	Search	Bird/Hail Strike
1/14/2026, 10:17:19 AM	Login	-
1/14/2026, 10:16:38 AM	Logout	-

- Damage Assessment Image Search



Upload Image

Click image to enlarge and mark defects

Station:

Section:

-- Select Section --

Click to upload an image

PNG, JPG, JPEG up to 10MB

Additional Instructions (Optional)

Cancel

Search

WEEK 12 - 15

IMPLEMENT

CHANGES FROM FIRST PROTOTYPE:

- LLM Change: Using Phi4:14b instead of Gemma3:12b (More Reasoning/Higher accuracy in extracting codes from manuals)

Gemma3:12b

AMM 12-40-04 (📌 Page 91) 12-40-04/201, Bird Strike Cleaning 📄 AMM/12__041.PDF - Page 528
AMM 27-51-00 (📌 Page 91) 27-51-00/201, Trailing Edge Flap System 📄 AMM/27__041.PDF - Page 1377
AMM 27-81-00 (📌 Page 91) 27-81-00/201, Leading Edge Flap System 📄 AMM/27__041.PDF - Page 2282
AMM 71-00-47 (📌 Page 91) 71-00-47/101, Power Plant - Trouble Shooting 📄 AMM/71_C_041.PDF - Page 766

phi4:14b

AMM 12-40-04 (📌 Page 91) 12-40-04/201, Bird Strike Cleaning 📄 AMM/12__041.PDF - Page 528
AMM 27-51-00 (📌 Page 91) 27-51-00/201, Trailing Edge Flap System 📄 AMM/27__041.PDF - Page 1377
AMM 27-81-00 (📌 Page 91) 27-81-00/201, Leading Edge Flap System 📄 AMM/27__041.PDF - Page 2282
AMM 71-00-47 (📌 Page 91) 71-00-47/101, Power Plant - Trouble Shooting 📄 AMM/71_C_041.PDF - Page 766
SRM 51-10-00 (📌 Page 94) SRM 51-10-00, Aerodynamic smoothness and damage investigation 📄 null - Page null

- VLM Change: Using qwen2.5-vl-7b instead of Gemma3:12b (More accurate in terms of extracting information from images)

Gemma3:12b

Defects: Scratch, Low Severity

qwen2.5-vl-7b

Defects: Paint-Off, Dent, Low Severity

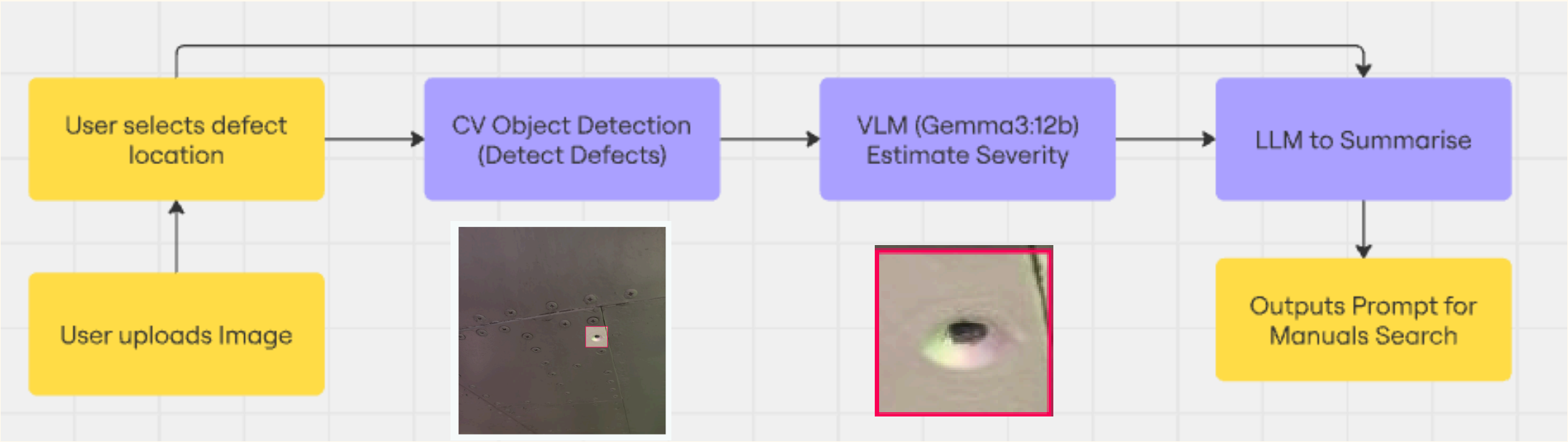
IMPLEMENT

CHANGES FROM FIRST PROTOTYPE:

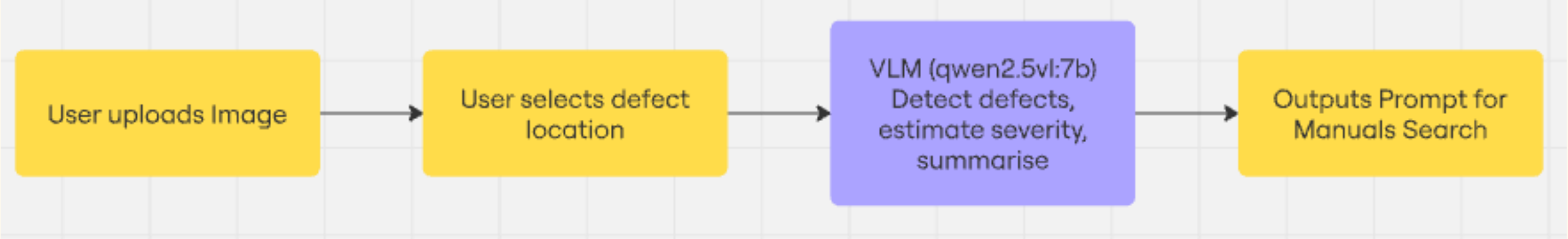
Used only VLM for the Damage Assessment feature:

Rather than combining it with a computer vision object detection model, relying solely on VLM achieves equal or better results in less time and enables classification of a wider range of defects, since object detection models are limited by the small number of defects they are trained on.

Before: VLM + LLM + CV



After: VLM ONLY



CHALLENGES FACED

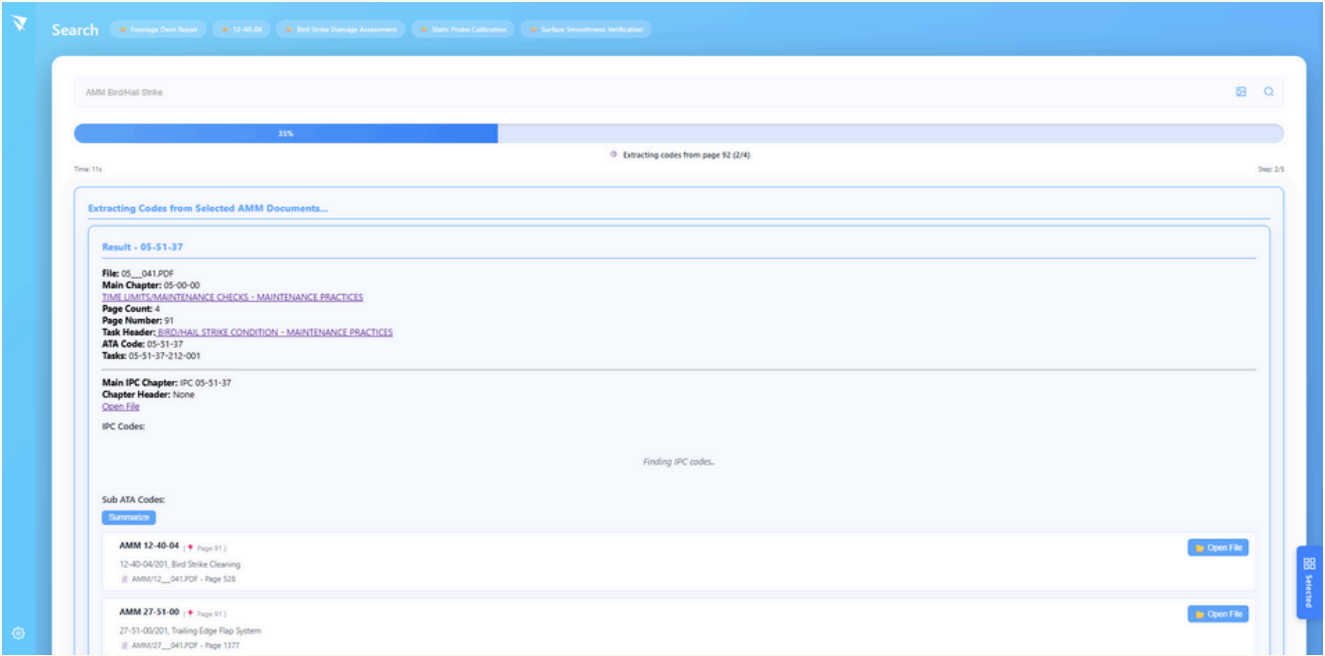
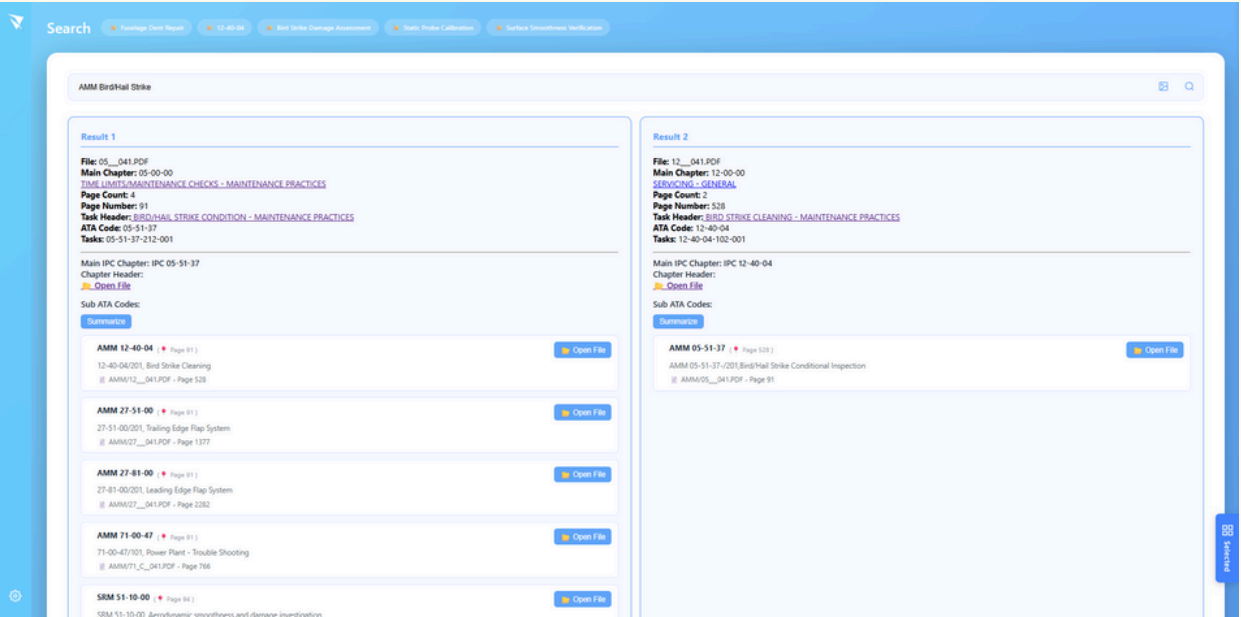
Problem	Description	Mitigation
Latency from LLM calls	Because the system relies on semantic understanding + LLM rather than simple keyword matching, response times can vary depending on the length and complexity of the manual section being processed. Longer chapters or multi-page queries may introduce noticeable delays compared to traditional keyword search engines.	Stream results via Flask API – Sub-codes are updated in real time as they are retrieved, ensuring the frontend reflects progress continuously.
LLM/Hardware Limitations	Since the solution currently runs on a locally hosted LLM (Ollama Phi4:14b) which contains 14 billion parameters and takes up a majority of computing power to run. It is bound to be inaccurate and slower compared to larger existing models like GPT-4 which contains an estimated 1.76 trillion parameters.	Chunk text uploads page by page – Documents are processed incrementally to prevent the LLM from experiencing the “Lost in the Middle” effect, maintaining accuracy across long inputs.

OPERATE

- **LIST OF ACTIVITIES CARRIED OUT IN OPERATE STAGE:**
 - Web Application was tested with a set of questions given by engineers to test out the accuracy of the model.
 - Other features, such as the Image Search, were also tested by the clients to gain a deeper understanding of the system’s capabilities. These tests evaluated the accuracy of image interpretation by the VLM and provided insights into the usability and overall UI/UX of the web application.

BENEFITS

- **HOW HAS THE PROJECT BENEFIT THE USERS?**
 - By enabling engineers to instantly access the exact manual and relevant sections, organizations reduce document search time by up to **10 times (from ~5mins to ~30secs)**, allowing maintenance teams to dedicate more effort to actual troubleshooting and repair tasks.
 - **Improved retrieval accuracy** by combining domain-specific chunking, clean pre-processing, and high-quality semantic embeddings.
 - All manuals and sections are accessible in one place, having **consolidated knowledge**. The dashboard can grow with new manuals, being **scalable** and supporting long-term operational needs.
 - Standardized retrieval ensures all specialists are working with the **same accurate information**. Validation is also required by specialists to ensure retrieved information is accurate.



POTENTIAL IMPROVEMENTS

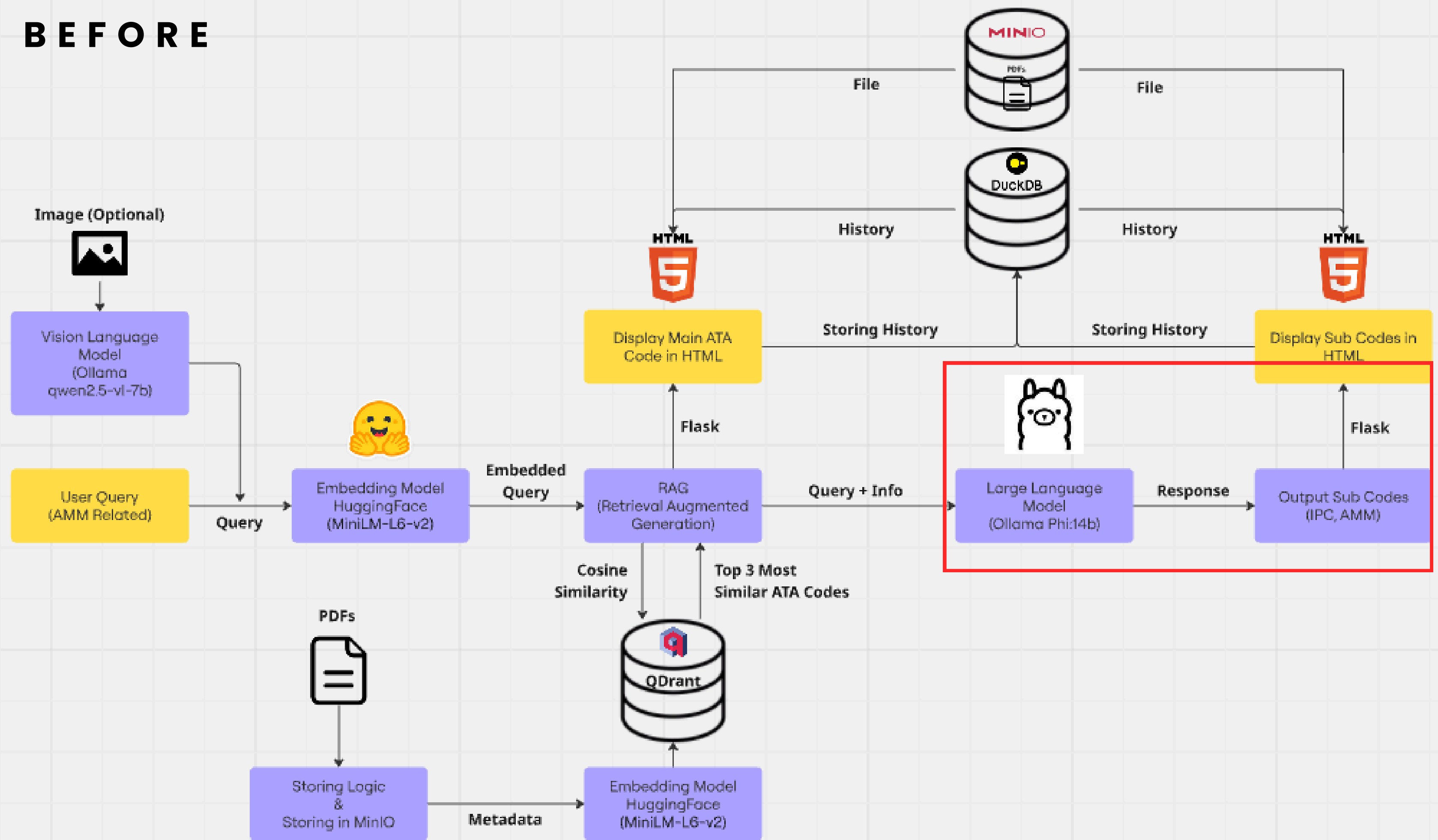
- **ADDING THE ATA SUB-CODES AS METADATA**

Integrating a Large Language Model (LLM) into preprocessing streamlines both retrieval and extraction of ATA sub-codes into a single step. This reduces processing time to under 5–10 seconds, improving efficiency without compromising accuracy.

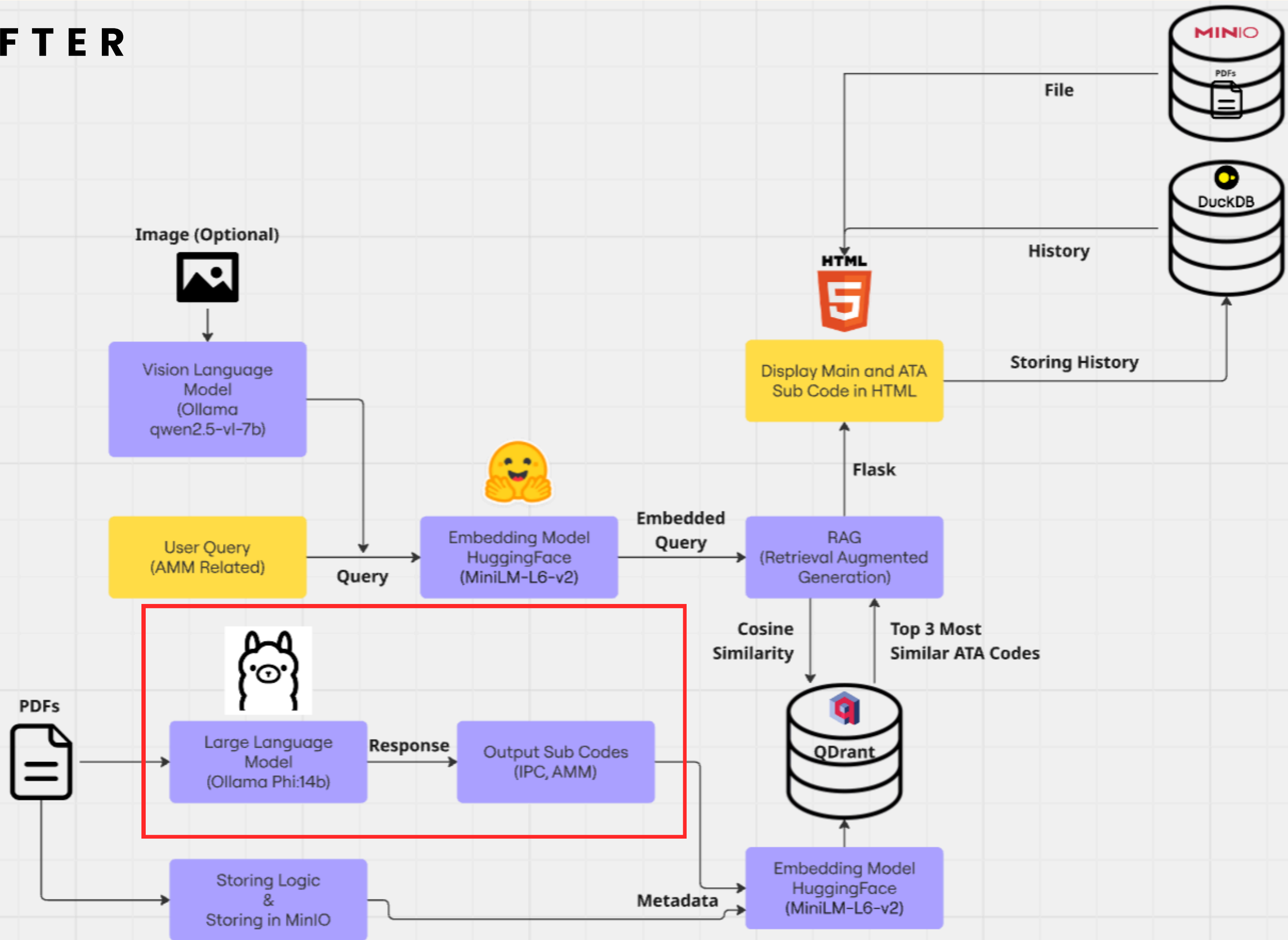
While integrating a LLM can shorten retrieval and extraction to under 5–10 seconds, it significantly increases preprocessing and storage time. Instead of taking one to two hours, preparing manuals such as AMM, IPC, and SRM could require one to two days. (Up to Client)

This extended preprocessing time is especially problematic for manuals that require frequent updates.

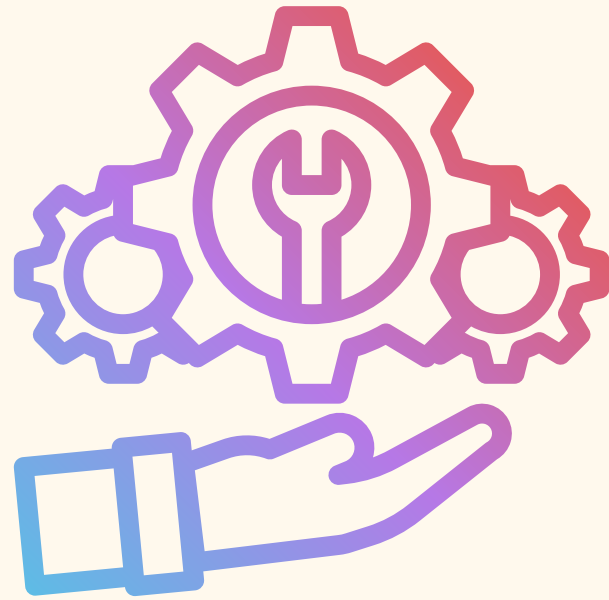
BEFORE



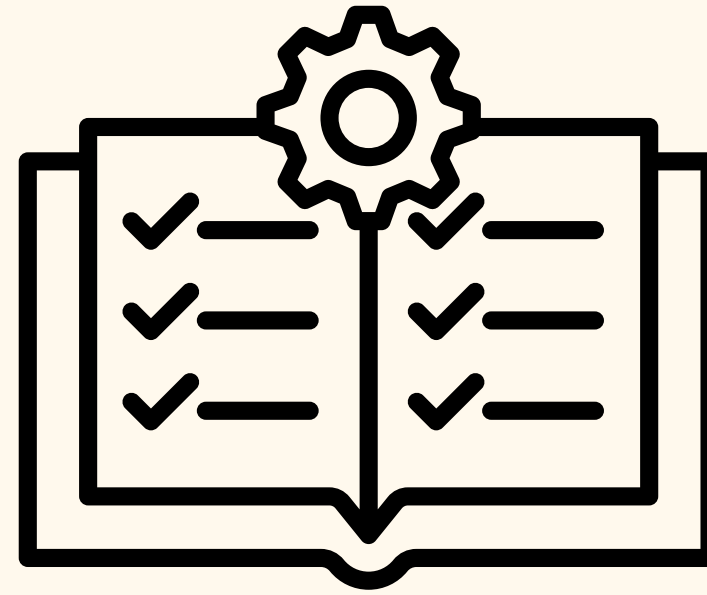
AFTER



FUTURE IMPROVEMENTS/INTEGRATIONS



PREDICTIVE MAINTENANCE

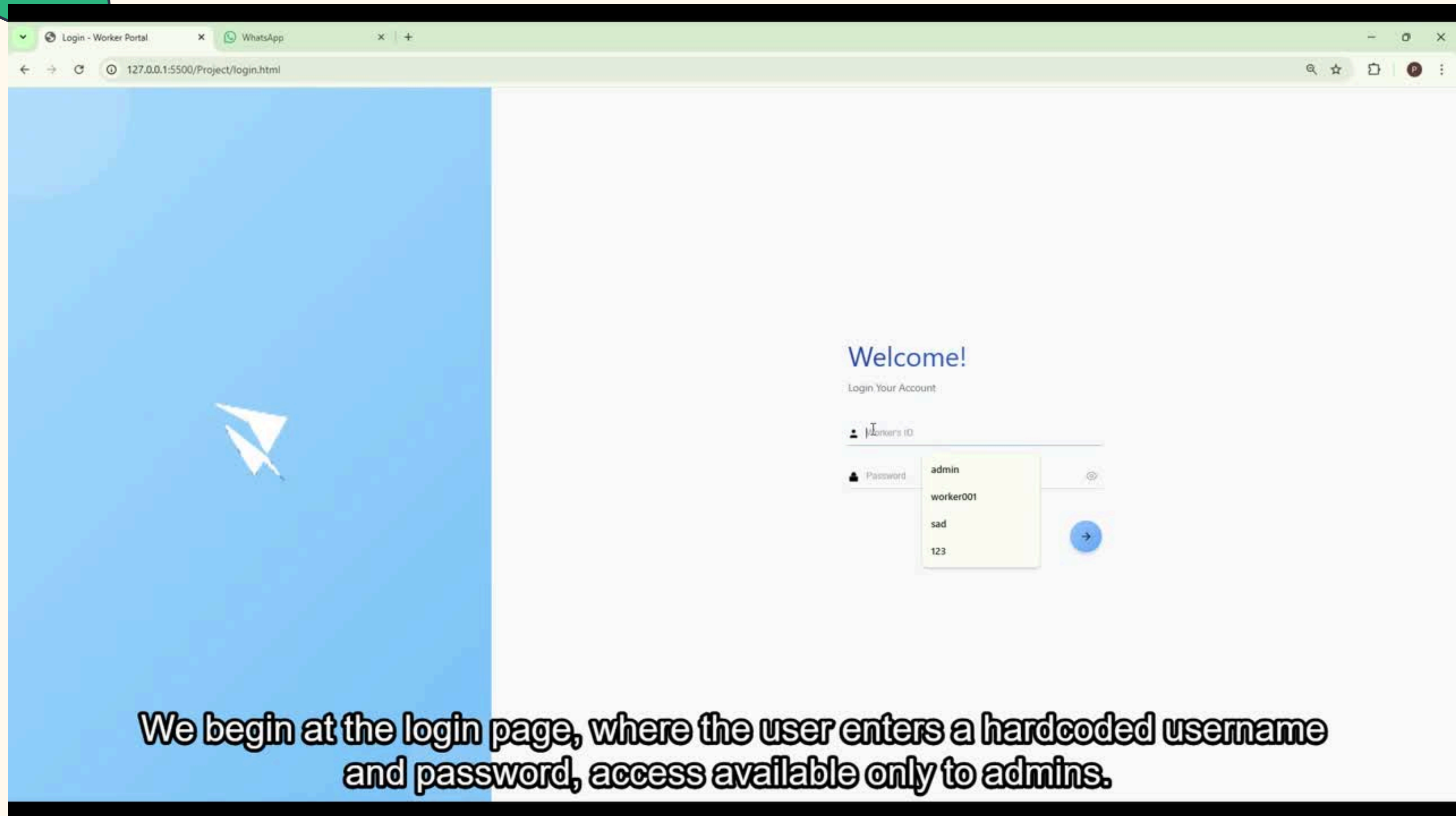


PARTS CATALOG



PRE-SCHEDULE

DEMO VIDEO



SKILLS LEARNED & APPLIED

• SOFTWARE SKILLS:

- Web Development (HTML/CSS/JS)
- Prompt Engineering and AI (LLM/VLM/Ollama)
- RAG (Retrieval Augmented Generation) Pipeline logic
- Worked with Different Types of Databases (Qdrant, DuckDB, MinIO)
- Computer Vision
- Frontend/Backend Development (FlaskAPI, Streaming)

• SOFT SKILLS:

- Self Learning & Problem Solving (Research, Trial & Error)
- Presentation (SP I&I, CA5, presenting to client)
- Time Management
- Adaptability (Trying out new logics/software)



A D D I T I O N A L P R O J E C T

CURRENT PROJECT - SMART AI ADVERTISER

- **PROBLEM STATEMENT**

How can the company achieve greater market popularity and optimize personalized recommendation services to enhance customer engagement?

- **SOLUTION**

A smart interactive agent act like a virtual avatar that provide personalised user experience with advertisement.

- **FEATURES**

CONVERSATIONAL CHATBOT

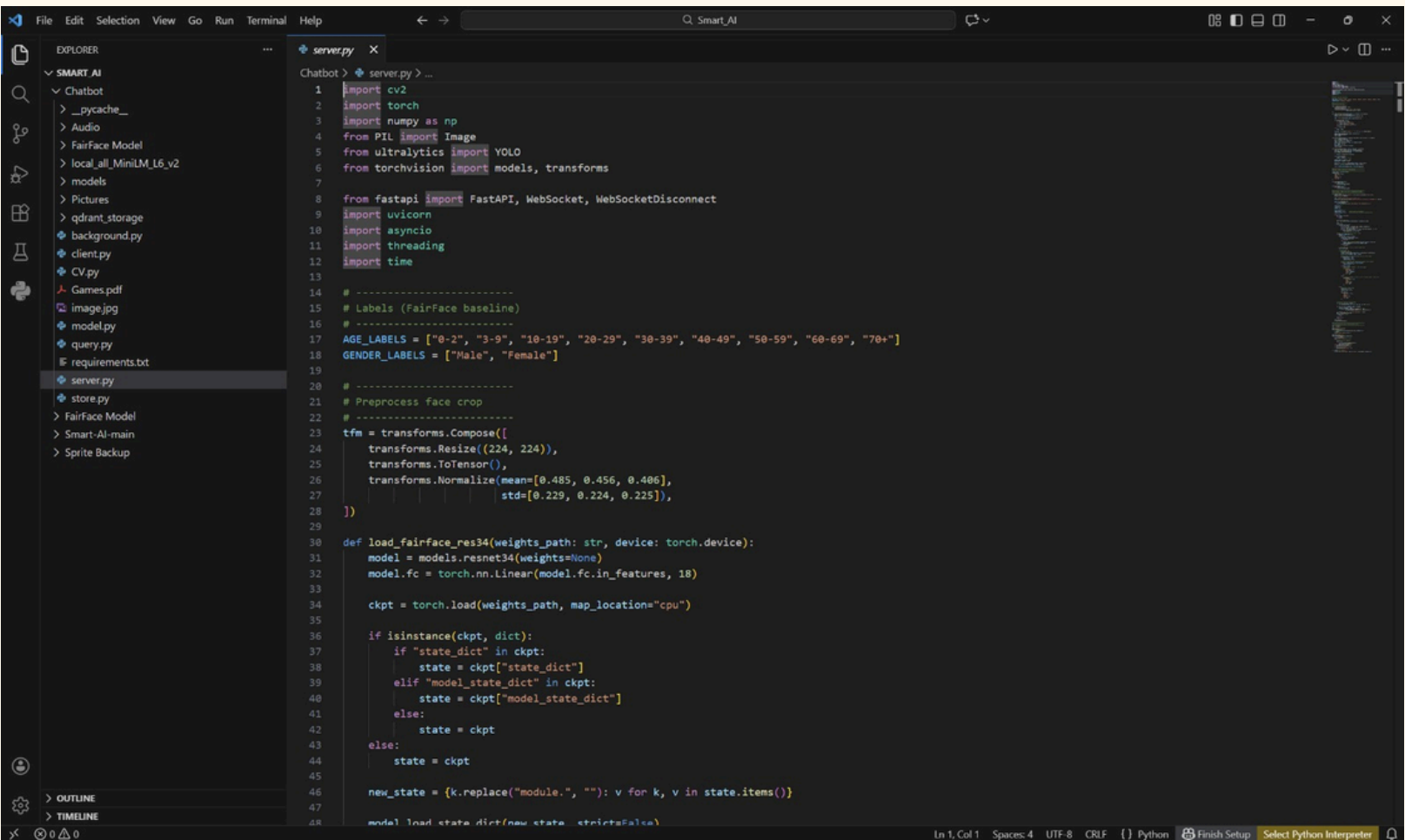
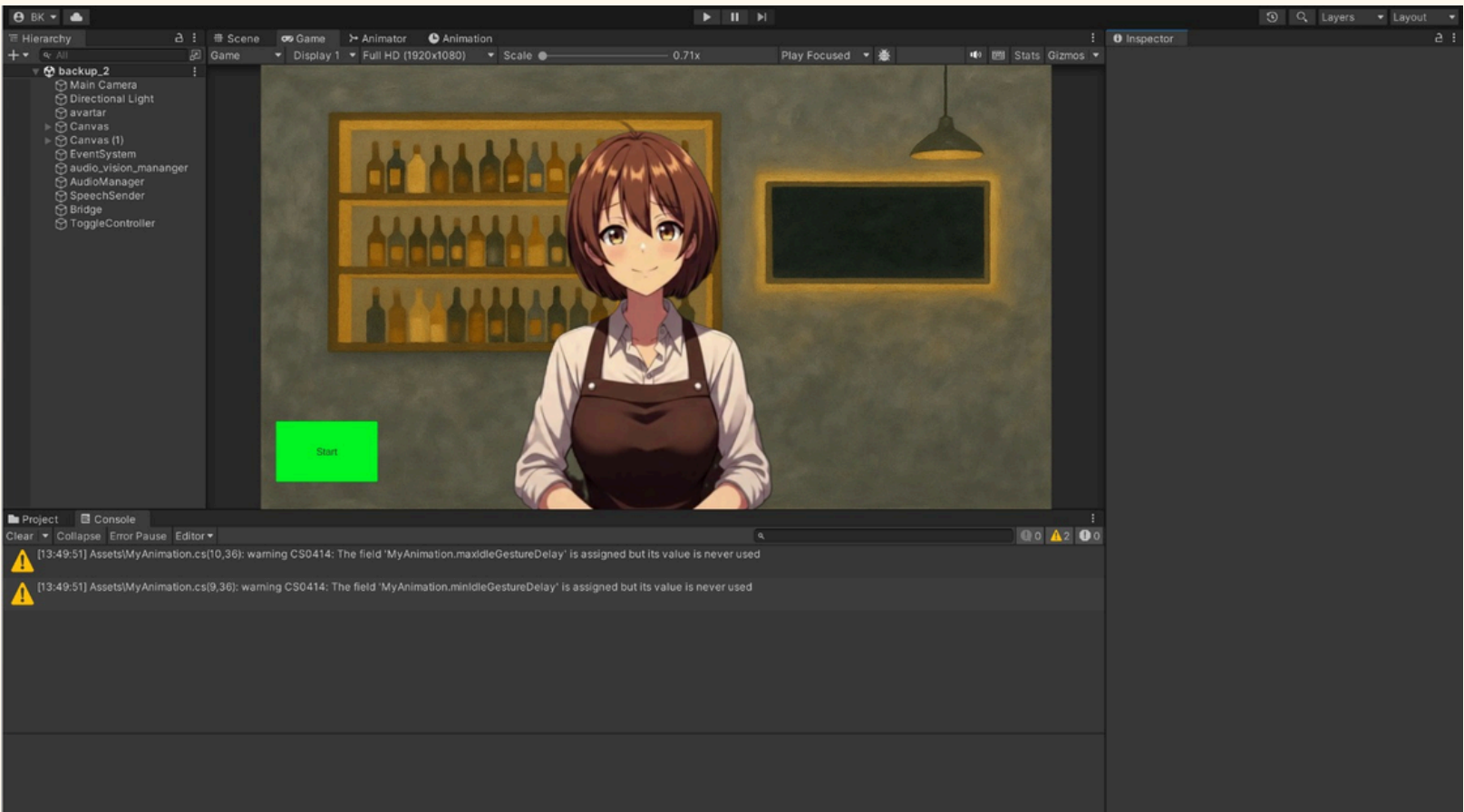
LLM (Large Language Model), ASR (Automatic Speech Recognition), and TTS (Text to Speech) create human-like chatbot interactions, boosting engagement and brand awareness through real-time personalization.

COMPUTER VISION

Computer vision detects presence, age, and gender to assign avatars and improve personalized recommendations.

RECOMMENDATIONS

RAG (Retrieval Augmented Generation) ensures precise, engaging information delivery for personalized recommendations and user support.



CURRENT PROJECT - SMART AI ADVERTISER

• FEATURE - COMPUTER VISION

Our Computer Vision uses YOLOv8 for real-time face detection and age/gender estimation. The system selects the largest bounding box to represent the primary user in the scene, ensuring stable and reliable results. The detected attributes are then sent to Unity via FastAPI, enabling seamless integration between the computer vision backend and the application frontend. These results can further be used to support personalized product recommendations by combining user attributes with Retrieval Augmented Generation (RAG), where relevant information is retrieved from a knowledge base and used to generate more context-aware and tailored recommendations.

Dataset + Model



FairFace Dataset

+



YoloV8

CURRENT PROJECT - SMART AI ADVERTISER

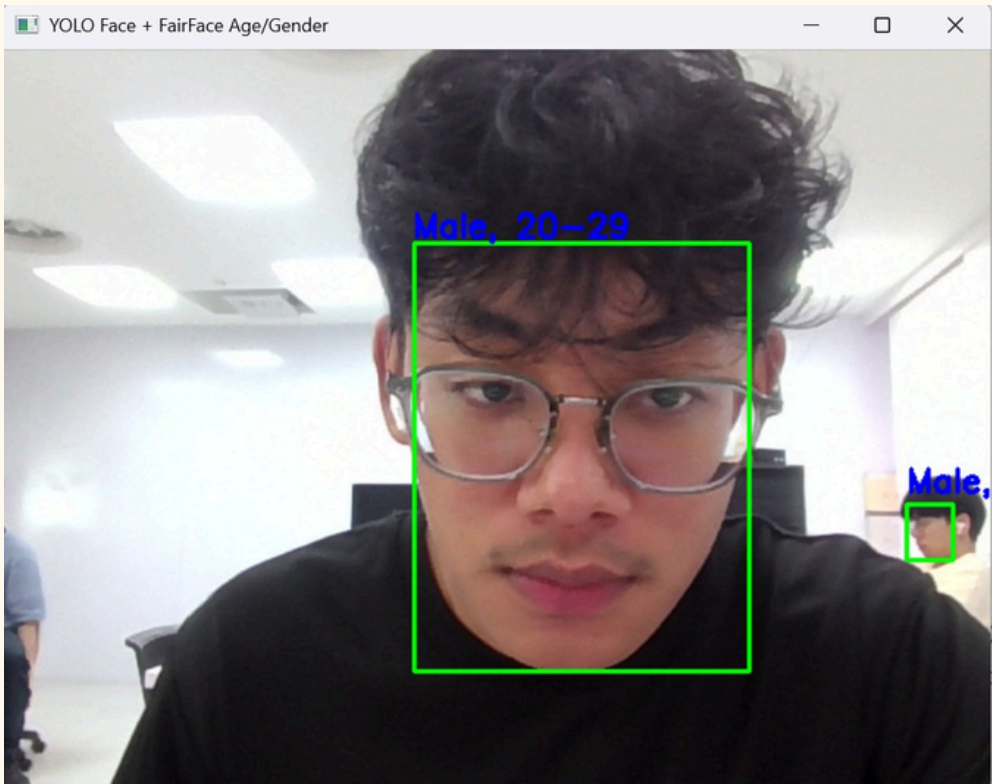
• FEATURE - COMPUTER VISION

Instead of relying solely on traditional computer vision to estimate age and gender, we experimented it together with a visual language model (VLM), specifically Ollama’s Gemma3:12b.

This approach allows us to infer a wider range of attributes beyond the predefined classes that computer vision models are typically trained on such as race, hair color, and other distinguishing features. By leveraging this set of information, we can deliver a more personalized product recommendation experience for users.

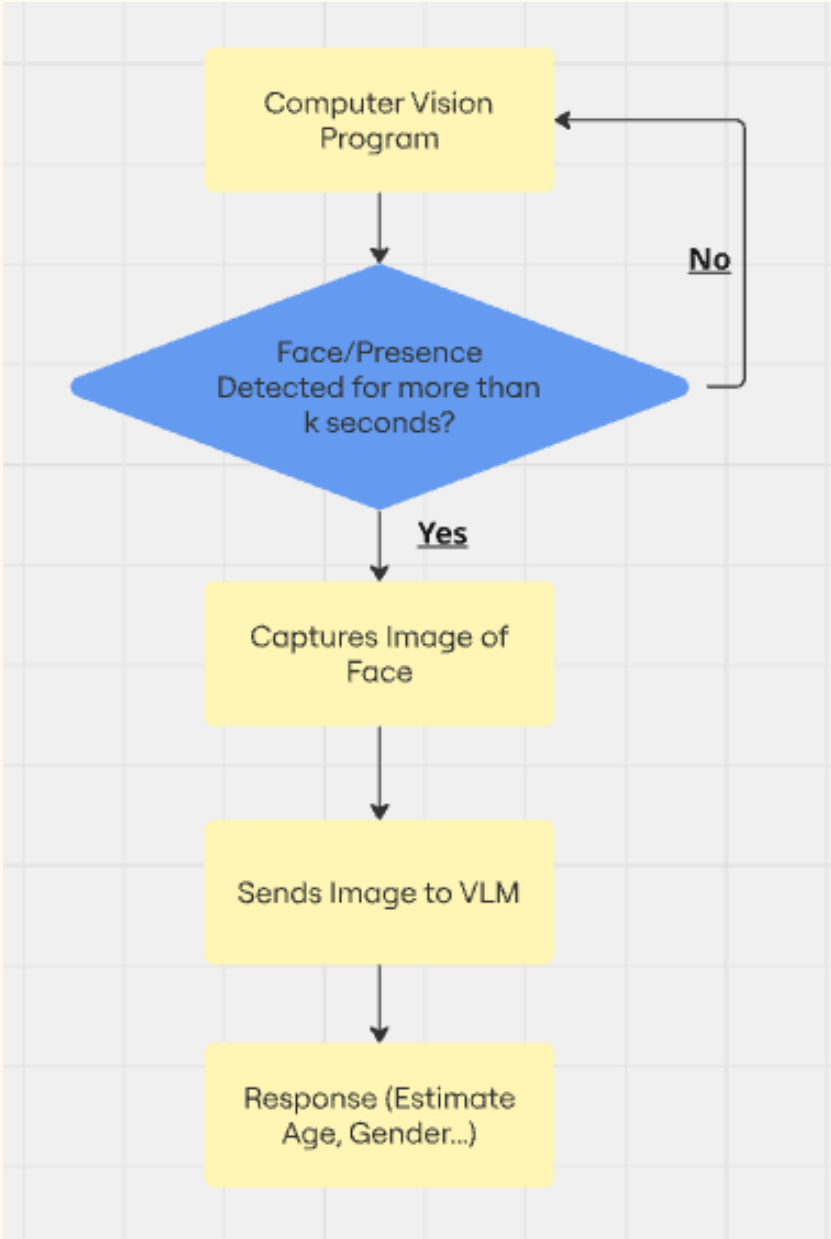
Computer Vision Results

Male, (20-29)



VLM Results

20-25, Male, Mature



CURRENT PROJECT - SMART AI ADVERTISER

• CREATING OF AVATAR ANIMATION

Input

- Character reference image (photo)
- Text prompt describing motion or expression

Process

- Character image is attached as a reference
- Prompt guides how the character should move or act
- AI model generates a video based on the image and prompt

Output

- Generated character video
- Maintains character appearance and style
- Video is later converted into frames for Unity

Overall WorkFlow:

**Character Image + Text Prompt → AI Video Generator
→ Character Video → Convert to Frames → Unity Animation**

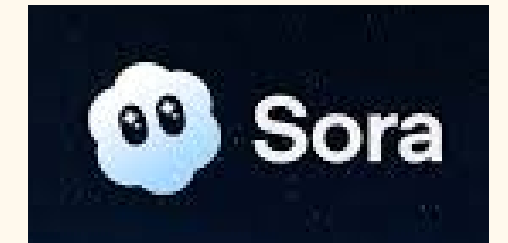
Reference Photo(AI generated)



Tools used for Video Generation



NanoBanana



Open-Sora

Tools used for Animation



Unity

CURRENT PROJECT - SMART AI ADVERTISER

- CREATING OF AVATAR ANIMATION



Video Generated

CURRENT PROJECT - SMART AI ADVERTISER

• CREATING OF AVATAR ANIMATION

Why convert into frames?

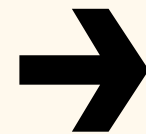
Purpose

- Convert generated video into image frames for use in Unity
- Image sequences are more flexible and Unity-friendly than video files

Method

- Python script using OpenCV
- Reads input video and extracts frames at a fixed rate (10 FPS)
- Saves frames sequentially as PNG images

Video Generated



Tools used for Animation



OpenCV

CURRENT PROJECT - SMART AI ADVERTISER

• FRAME - TO - ANIMATION SETUP IN UNITY

Process

- Extracted image frames are imported into Unity
- Frames are grouped and converted into animation clips
- Each animation clip represents a character action

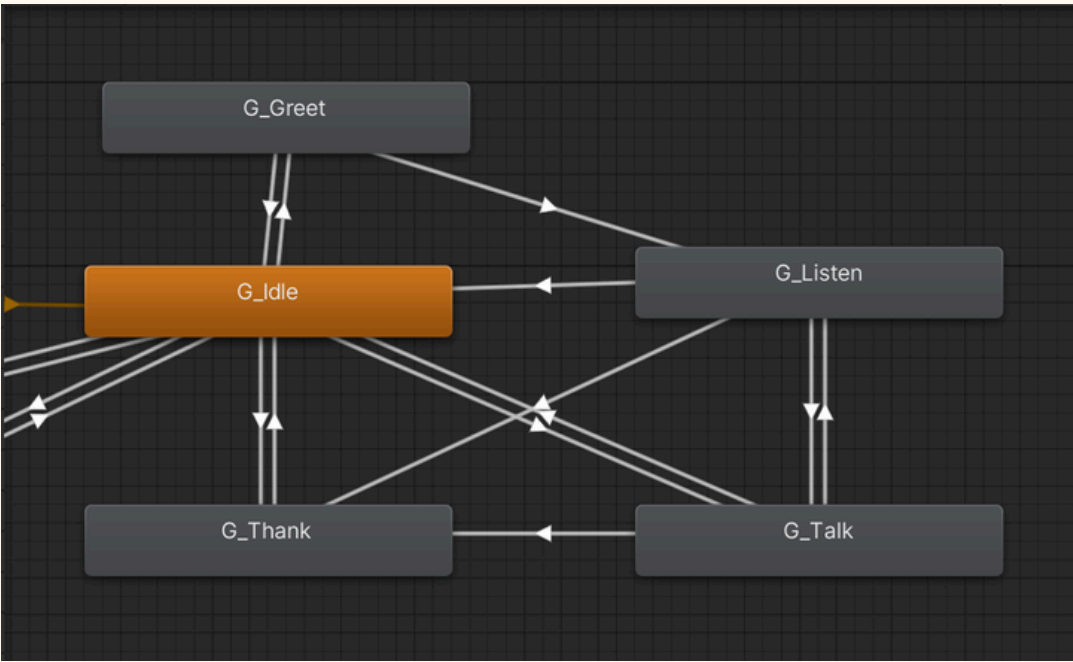
Animations Created

- G_Idle – Default waiting state
- G_Greet – Greeting animation
- G_Listen – Listening state
- G_Talk – Talking animation
- G_Thank – Thank-you animation



Animator Control

- Unity Animator is used to manage transitions
- Character switches animations based on interaction flow
- Smooth transitions ensure natural character behavior



CURRENT PROJECT - SMART AI ADVERTISER

• FRAME - TO - ANIMATION SETUP IN UNITY

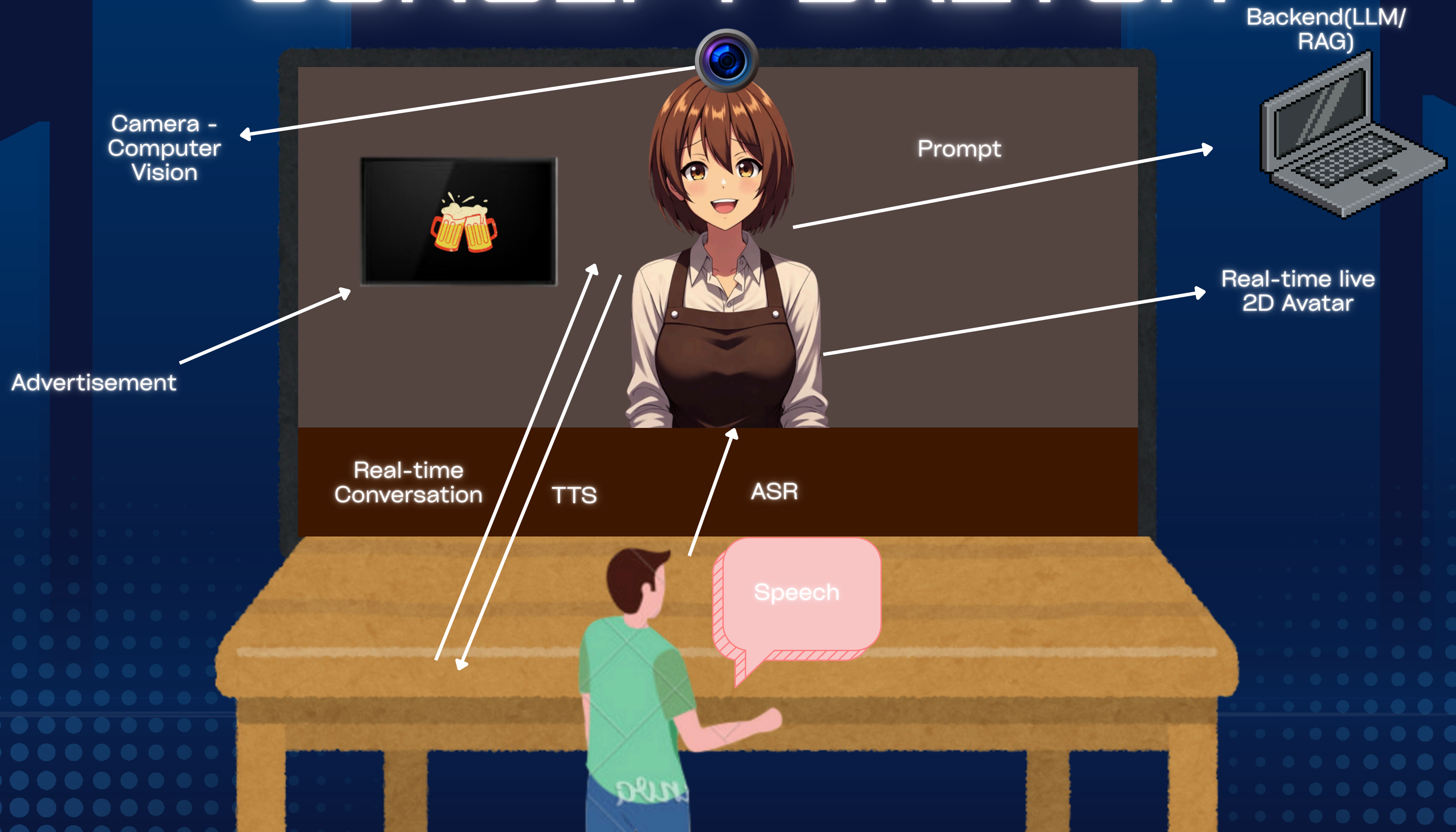
Avatar Behaviour Logic

- Idle State
 - Avatar remains idle when no face is detected by the camera
- Greeting
 - When a face is detected, the avatar starts greeting the user
- Listening
 - Avatar listens to the user's speech input
- Conversation
 - Avatar engages in back-and-forth conversation with the user
- Farewell
 - When the user says goodbye, the avatar thanks the user
- Return to Idle
 - Avatar returns to idle state and waits for the next user

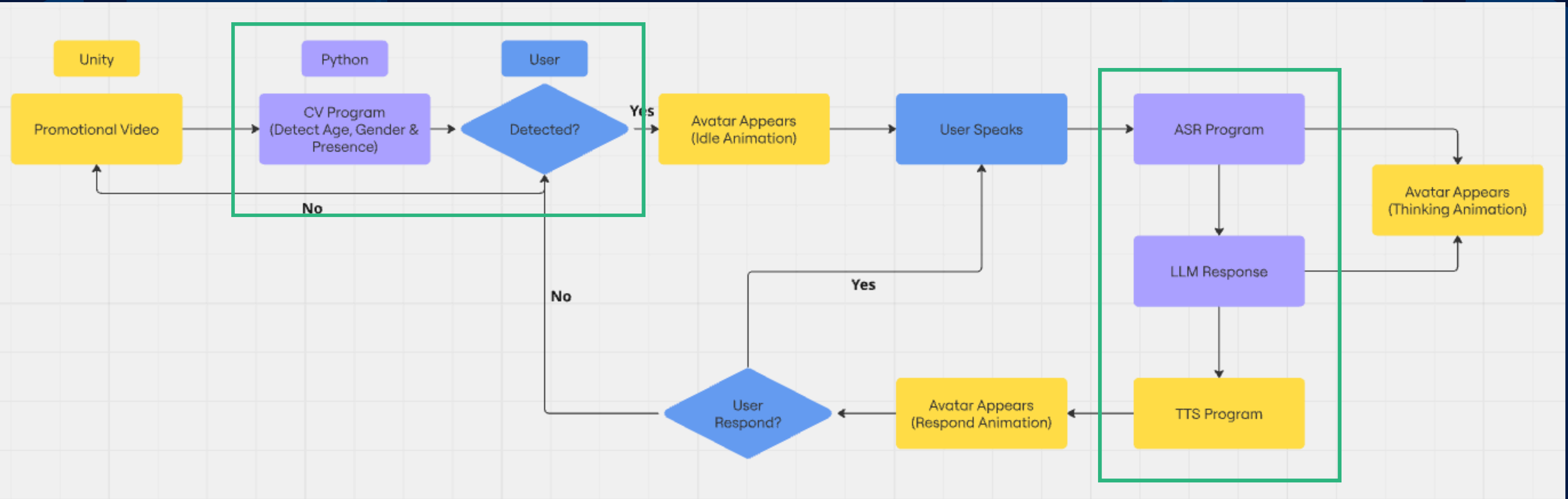




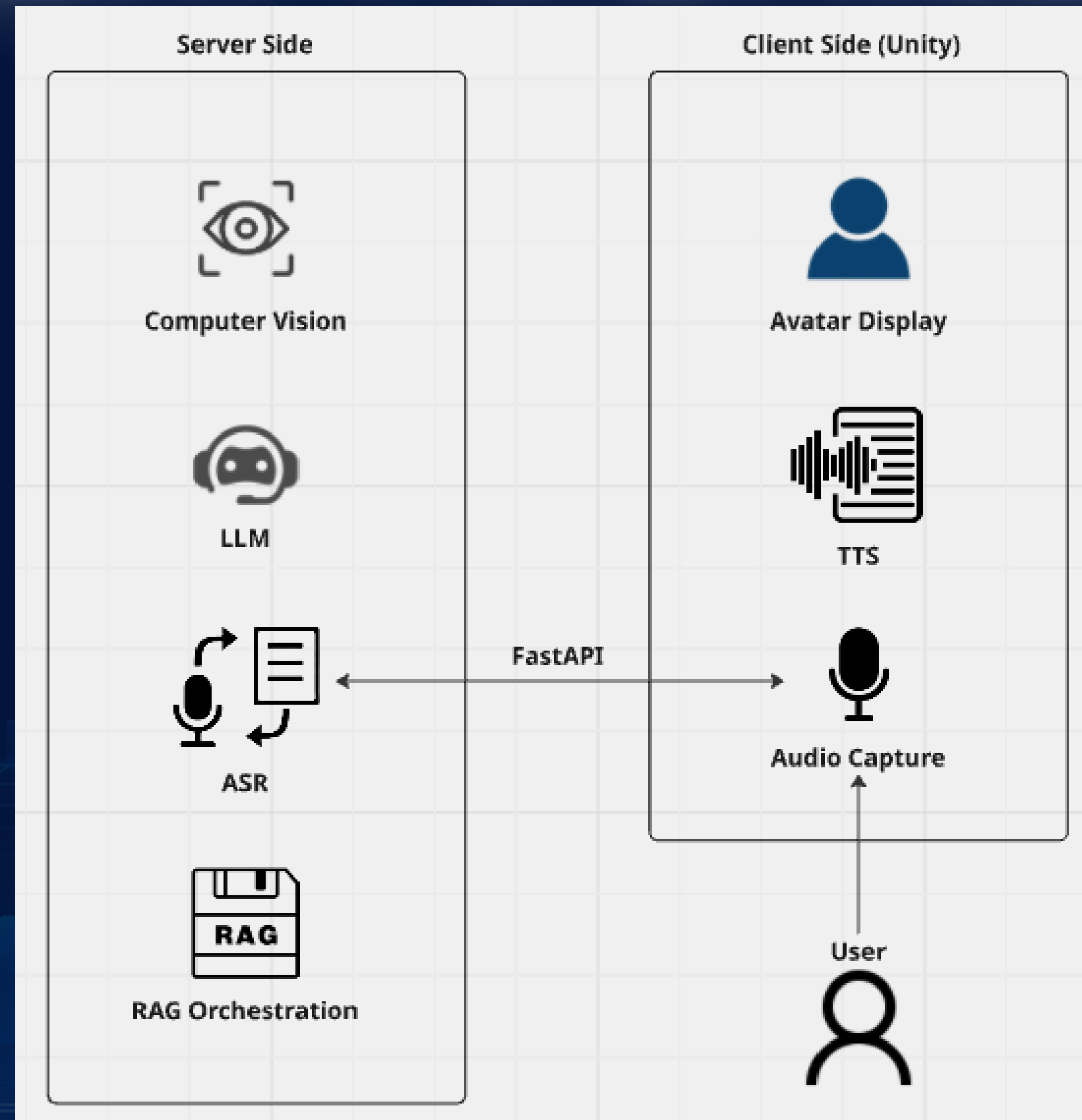
CONCEPT SKETCH



Data Flow



System Architecture



REFLECTION

During my work on the web application, I applied knowledge I gained in school, such as information retrieval, structured data handling, and systems/prompt engineering. Since it's my first time working with RAG, building a RAG pipeline that extracts ATA codes from manuals for aircraft engineers was quite a refreshing feeling. I think I did well in integrating multiple PDFs, improving retrieval accuracy, and building the pipeline in general.

Although I initially struggled with inconsistent accuracy, storing metadata and long search time. Looking ahead, I aim to refine chunking logic, complete the HTML migration, and improve retrieval so that engineers receive more precise, reliable results.

KEY LEARNINGS

• SP I&I



KEY LEARNINGS

- SINGTEL VISIT



- 5G&AIOT CENTRE



REFERENCES

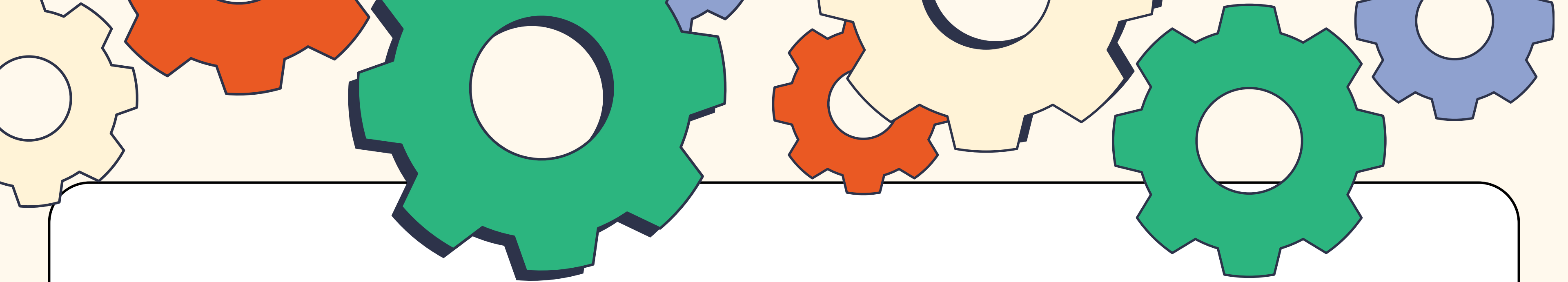
Mikko Lietsalmi (2025). Trends and workforce challenges in aircraft maintenance. [online] Qoco.aero. Available at: <https://www.qoco.aero/blog/trends-and-workforce-challenges-in-aircraft-maintenance>.

2Lyons Aerospace (2025). Aviation maintenance plays a pivotal role in ensuring that aircraft are safe, reliable, and efficient for flight. However, despite technological advancements and rigorous regulations, aviation maintenance still faces several significant challenges. [online] Linkedin.com. Available at: <https://www.linkedin.com/pulse/what-most-common-challenges-aviation-maintenance-2lyonsaerospace1-opmvc/>.

RecordOwl (2024). AVOTRON PTE. LTD. [online] RecordOwl. Available at: <https://recordowl.com/company/avotron-pte-ltd> [Accessed 10 Feb. 2026].

DeepAI. (2019). Cosine Similarity. [online] Available at: <https://deepai.org/machine-learning-glossary-and-terms/cosine-similarity>.

TensorFlow. (n.d.). Word embeddings | Text. [online] Available at: https://www.tensorflow.org/text/guide/word_embeddings.



THANK YOU